

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский ядерный университет «МИФИ»

Обнинский институт атомной энергетики –

филиал федерального государственного автономного образовательного учреждения высшего
образования «Национальный исследовательский ядерный университет «МИФИ»
(ИАТЭ НИЯУ МИФИ)

Отделение	<u>Интеллектуальные кибернетические системы</u>
Направление подготовки	<u>Информатика и вычислительная техника</u>

Научно-исследовательская работа

Кодогенератор для работы с CANopen

Студент группы ИВТ-Б22 _____ Карасев Н. А.

Руководитель
инженер-программист _____ Жильцов Д. И.

Обнинск, 2026 г

РЕФЕРАТ

Работа 81 стр., 1 табл., 4 рис., 16 ист.

Ключевые слова: CAN, CANOPEN, HASKELL, RUST, ПРОМЫШЛЕННАЯ АВТОМАТИЗАЦИЯ.

Объектом исследования является процесс построения программного интерфейса для работы со словарём объектов CANopen. Предметом исследования выступает архитектура кодогенератора, преобразующего описание словаря объектов и дополнительные интерпретационные файлы в типизированные модули языка Rust.

Цель работы состоит в разработке кодогенератора на языке Haskell для автоматического построения Rust-интерфейса к словарю объектов CANopen. Для достижения этой цели проанализирована предметная область, выявлены требования к генератору, разработана поэтапная архитектура преобразования данных и реализована соответствующая система.

В работе предложен конвейер из четырёх основных этапов: инициализации конфигурации, синтаксического разбора входных форматов, компиляции промежуточного представления и перевода в язык Rust. Показано, что на уровне архитектуры каждый этап реализует отдельное отображение между формальными представлениями данных и обладает собственными инвариантами.

Результатом работы является реализованный кодогенератор, автоматически формирующий Rust-модуль по формальному описанию словаря объектов CANopen и набору вспомогательных конфигурационных файлов. В процессе работы система выделяет требуемую часть словаря, строит промежуточную прикладную модель объектов и затем генерирует итоговый модуль с типами данных, trait-реализациями, аннотирующими типами Index, Value, Dict и необходимыми импортами. На примере секции Info и словаря Maxon показано, что система поддерживает повторное использование общих фрагментов словаря, прикладную интерпретацию значений и формирование целостного типизированного интерфейса.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 Предметная область	8
1.1 Словарь объектов CANopen как источник исходных данных . .	8
1.2 Уровни интерпретации данных словаря объектов	9
1.3 Требования к системе	10
2 Архитектура проекта	12
2.1 Параметры входных данных	12
2.2 Общая архитектура	13
2.3 Монадическая архитектура этапов проекта	15
3 Этап инициализации	17
3.1 Назначение инициализации	17
3.2 Архитектура инициализации	17
3.3 Типы Desc и BuildUnit	18
3.4 Разбор аргументов и выбор набора дескрипторов	19
3.5 Парсинг формата desc	19
3.6 Разрешение зависимостей и проверка ограничений	20
3.7 Результат этапа инициализации	21
4 Этап синтаксического разбора	22
4.1 Назначение синтаксического разбора	22
4.2 Архитектура синтаксического разбора	22
4.3 Границы этапа	23
4.4 Комбинаторные парсеры как абстрактная вычислительная модель	24
4.5 Тип Ind	27
4.6 Парсинг словаря объектов EDS	27
4.7 Парсинг фильтра FLT	28
4.8 Парсинг регистровых интерпретаций REG	28
4.9 Результат этапа Raw	30
5 Компиляция промежуточного представления	31

5.1	Назначение компиляции промежуточного представления	31
5.2	Архитектура компиляции промежуточного представления . . .	32
5.3	Нормализация фильтра	33
5.4	Декодирование словаря объектов	34
5.5	Декодирование регистровых описаний	35
5.6	Склейка словаря и регистровой карты	37
5.7	Свёртка в законченные объекты	38
5.8	Результат этапа Compile	39
6	Перевод в Rust и генерация выходного кода	41
6.1	Назначение перевода в Rust	41
6.2	Архитектура перевода в Rust	42
6.3	Модель языка Rust	45
6.4	Двухуровневый переводчик	48
6.4.1	Назначение	48
6.4.2	Архитектура уровней	49
6.4.3	Почему существует RustDesc	50
6.4.4	Интерпретация Rust-типа	51
6.4.5	Задание черт	52
6.4.6	Задание атрибутов	52
6.5	Аннотирующие типы	53
6.5.1	Назначение	53
6.5.2	Архитектура генерации аннотирующих типов	53
6.5.3	Секции и словари	54
6.6	Генерация импортов	55
6.7	Вывод модуля	57
7	Пример работы генератора	59
	ЗАКЛЮЧЕНИЕ	61
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	63
	Приложение А	65

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящем отчете о НИР применяют следующие термины с соответствующими определениями:

Controller Area Network (CAN) — шина обмена сообщениями.

CANopen — протокол связи поверх CAN-шины.

Object Dictionary (OD) — объектный словарь.

Command Line Interface (CLI) — интерфейс командной строки.

Electronic Data Sheet (EDS) — файл описания словаря объектов CANopen.

Service Data Object (SDO) — механизм сервисного доступа к объектам словаря в CANopen.

Process Data Object (PDO) — механизм обмена процессными данными в CANopen.

Application Programming Interface (API) — программный интерфейс доступа к функциям и данным системы.

FLT — вспомогательный файл, задающий область генерации в виде белого или чёрного списка индексов.

REG — вспомогательный файл, задающий правила представления объектов словаря в прикладных типах.

DESC — файл-дескриптор единицы генерации, связывающий входные данные, тип модуля и параметры вывода.

ВВЕДЕНИЕ

Промышленное и встраиваемое программное обеспечение регулярно работает с устройствами, обмен данными с которыми строится на основе CAN и CANopen. В таком контексте разработчику требуется не только понимать структуру словаря объектов устройства, но и иметь удобный программный интерфейс, исключающий постоянную ручную работу с индексами, подындексами и двоичными представлениями полей. CANopen определяет прикладной слой, коммуникационный профиль и сетевые сервисы взаимодействия узлов, а словарь объектов является центральной точкой описания их параметров, адресации, типов и режимов доступа [1—3].

В типичной сети CANopen один управляющий узел выполняет роль мастера, а один или несколько узлов выступают ведомыми устройствами. В этой конфигурации словарь объектов служит общей точкой описания как для программ на стороне мастера, так и для программ самих ведомых устройств. Поэтому разработанный кодогенератор применим как для формирования кода управляющей стороны, работающей со словарями удалённых узлов, так и для генерации типизированного представления словаря внутри программного обеспечения конкретного CANopen-устройства [maxon_epos4_control_2024; 1].

Ручное описание такого интерфейса на языке прикладной разработки удобно только для небольших и статичных словарей. При появлении десятков и сотен записей, массивов, структур, перечислений и специализированных соглашений о сериализации такое описание становится серьёзной проблемой. Для её решения в рассматриваемом проекте реализован кодогенератор на языке Haskell, который читает словарь объектов и набор дополнительных файлов интерпретации, а затем строит прикладные Rust-модули.

Актуальность темы определяется тем, что подобный инструмент уменьшает объём однообразного ручного кода, централизует правила интерпретации объектов и снижает риск ошибок при работе с битовыми полями, перечислениями и сложными типами устройств. Для словарей CANopen это особенно существенно, поскольку один и тот же объект может участвовать одновременно в сетевом протоколе, прикладной логике и преобразовании низко-

уровневых значений в предметно-значимые структуры [4; 5].

Целью работы является создание кодогенератора на языке Haskell для работы с CANopen.

Для достижения поставленной цели в работе решаются следующие задачи:

1. анализ предметной области CANopen и роли словаря объектов;
2. выявление требований к кодогенератору;
3. создание общей архитектуры и этапов преобразования данных;
4. реализация данной архитектуры в коде;
5. демонстрация работы генератора на реальных примерах.

1 Предметная область

1.1 Словарь объектов CANopen как источник исходных данных

CANopen представляет собой профиль взаимодействия устройств в сети CAN, в котором основные параметры узла описываются через словарь объектов. Каждый объект адресуется парой «индекс – подындекс», а доступ к его значениям и обмен данными, связанными с ним, осуществляются через стандартные механизмы протокола, прежде всего SDO и PDO [1—3; 6]. В практических реализациях словарь объектов содержит как простые значения, так и составные записи: массивы, записи с подындексами, поля конфигурации, статусы, управляющие слова и диагностические параметры [5].

Для автоматизированной обработки такого словаря обычно используется файл EDS, где каждая запись описывается набором пар «ключ – значение». Однако одного файла EDS недостаточно для генерации удобного прикладного интерфейса. Формально словарь объектов задаёт структуру данных, но не отвечает на вопросы о том, какие биты в составе целочисленного значения следует трактовать как логические флаги, какие диапазоны формируют перечисление, а какие просто являются целочисленными значениями.

Дополнительную сложность представляет повторное использование общих фрагментов словаря. Разные устройства одного класса нередко содержат совпадающие группы объектов, отражающие общие части профиля устройства, стандартные служебные записи или единообразные настройки. На уровне самого EDS такие фрагменты обычно дублируются, однако при построении программного интерфейса это приводит к повторению одних и тех же типов, правил преобразования и элементов API.

Следовательно, задача кодогенерации в данной предметной области состоит не только в механическом переносе записей EDS в код, но и в построении типизированного интерфейса, который учитывает дополнительные правила интерпретации значений и допускает повторное использование общих частей словаря.

1.2 Уровни интерпретации данных словаря объектов

При работе со словарём объектов полезно различать несколько уровней интерпретации одних и тех же данных. На самом нижнем уровне объект словаря представляет собой элемент протокольного описания: ему сопоставлены индекс, подындекс, тип данных, атрибуты доступа и, возможно, значение по умолчанию. Именно этот уровень фиксируется в EDS и является исходной формой представления словаря.

Однако для прикладной разработки такого описания недостаточно. Значительная часть объектов CANopen на уровне протокола задаётся как целочисленные значения фиксированной разрядности, но в предметном смысле эти значения имеют более сложную структуру. Отдельные биты могут обозначать логические признаки, группы битов — коды состояний, а некоторые значения должны трактоваться как элементы конечного множества допустимых режимов. Иначе говоря, между протокольным представлением объекта и его прикладным смыслом существует дополнительный слой интерпретации.

С практической точки зрения можно выделить три уровня. Первый уровень — это *словарный*, на котором объект рассматривается как запись словаря объектов с адресацией и базовыми атрибутами. Второй уровень — *уровень прикладной семантики*, на котором значения получают более содержательное представление в виде перечислений, структур битовых полей, массивов или специализированных типов. Третий уровень — *уровень программного интерфейса*, на котором отдельные типы объединяются в согласованный API для чтения, записи и преобразования значений словаря.

Для кодогенератора принципиально важно поддерживать все три уровня одновременно. Если ограничиться только первым уровнем, результатом будет механическое отображение EDS в набор слабо связанных определений. Если учитывать только второй уровень, но не строить целостный интерфейс, разработчик по-прежнему будет вынужден вручную связывать конкретные типы с индексами словаря и операциями доступа. Поэтому генератор должен, с одной стороны, сохранять связь с исходной адресацией CANopen, а с другой — поднимать описание до уровня типобезопасного прикладного интерфейса.

Именно такое понимание уровней интерпретации объясняет появление в проекте дополнительных входных описаний помимо EDS, а также аннотирующих типов `Index`, `Value` и `Dict`. Они не дублируют словарь объектов, а выражают его на разных уровнях абстракции: от адресуемого элемента протокола до унифицированного программного интерфейса.

1.3 Требования к системе

К кодогенератору были предъявлены следующие требования:

- обработка EDS, что включает:
 - корректное отображение объектов и подобъектов словаря в типы и определения языка Rust;
 - возможность выбирать подмножество объектов и подобъектов, участвующих в генерации;
- автоматическое создание интерфейсов для работы со словарём как с единым объектом, а именно трёх сущностей:
 1. `Value` – суммарного типа, объединяющего значения всех поддерживаемых объектов словаря в едином типизированном представлении;
 2. `Index` – перечисления имён объектов словаря, связанного с их адресацией в терминах CANopen-индекса и подиндекса;
 3. `Dict` – структуры словаря, содержащей все сгенерированные объекты и предоставляющей унифицированный интерфейс чтения и записи по элементам `Index` и `Value`;
- возможность автоматизированной сериализации и десериализации объектов и подобъектов, при которой отдельный объект или его часть могут интерпретироваться как:
 1. перечисление;
 2. структура, каждое поле которой может быть:
 - (а) булевым значением;

- (b) целочисленным значением;
 - (c) перечислением;
- автоматическое наделение сгенерированных Rust-типов необходимыми trait-реализациями, включая:
 - реализацию trait-ов, определяемых природой объекта;
 - реализацию преобразований, необходимых для сериализации и десериализации;
 - реализацию trait-ов, связывающих конкретные типы объектов с интерфейсами Value, Index и Dict;
- генерация необходимых Rust-импортов, обеспечивающих корректную компиляцию выходного файла как самостоятельного модуля;
- возможность выделять общие части словаря в отдельные модули и повторно использовать их при генерации нескольких словарей.

2 Архитектура проекта

2.1 Параметры входных данных

Основным источником информации для работы программы является файл EDS. Однако одного этого описания недостаточно для задания всех параметров генерации. Система также должна позволять выбирать подмножество интерпретируемых объектов. Задавать такое подмножество непосредственно в EDS нецелесообразно, поскольку это означало бы модификацию файла, выступающего каноническим описанием словаря и обычно формируемого внешними средствами. Поэтому в архитектуру был введён отдельный входной формат, задающий подмножество интерпретируемых объектов и подобъектов. Далее этот файл обозначается как FLT или *.flt.

Аналогичное решение было принято для задания правил сериализации и десериализации объектов. Для этого был выделен формат REG или *.reg. Его название связано с тем, что в реализации на Haskell соответствующие описания интерпретируются как регистровые карты.

Может показаться, что фильтрацию и задание правил интерпретации можно было бы объединить в одном формате. Однако эти параметры относятся к различным уровням описания. Фильтр задаёт отображение на подмножество объектов словаря, не затрагивая их внутреннюю структуру. Напротив, REG-файл работает с конкретными объектами и задаёт правила их внутренней интерпретации, не оперируя множеством объектов как целым. Разделение этих функций на два формата делает архитектуру более строгой и в дальнейшем упрощает расширение пользовательского интерфейса.

Тем самым уже на входе возникают три различных файла. Кроме того, проект должен поддерживать повторное использование общих частей словаря в виде отдельных модулей, а эта задача лежит на уровне, внешнем по отношению к самому OD. Для её решения вводится файл-дескриптор DESC. Он задаёт единицу генерации: определяет, откуда берутся входные данные, как они связаны между собой, как называется генерируемый модуль, каков его тип, какими секциями он пользуется и куда должен быть выведен итоговый код. Иными словами, дескриптор связывает инфраструктурную конфигура-

цию с предметной моделью словаря объектов. В каждом дескрипторе фиксируется роль модуля: либо *Dictionary*, то есть полноценный словарь, либо *Section*, то есть общая часть словаря, используемая несколькими словарями и не обладающая самостоятельной завершённостью.

Для удобства анализа входные файлы и их роль в системе сведены в таблицу 1.

Таблица 1 – Назначение входных форматов проекта

Файл	Назначение
.desc	Описание единицы генерации, путей к файлам и зависимостей между модулями
.eds	Базовое описание словаря объектов CANopen
.flt	Белый или чёрный список индексов, участвующих в генерации
.reg	Правила интерпретации объектов как структур, перечислений и атрибутированных записей

Такое распределение обязанностей делает систему расширяемой. Например, новые правила интерпретации можно развивать на уровне REG-формата, не меняя архитектурный каркас проекта. В этом и состоит одно из достоинств выбранной декомпозиции: отдельные уровни модели можно усложнять независимо друг от друга.

2.2 Общая архитектура

Архитектуру приложения целесообразно рассматривать как композицию двух уровней:

1. верхнего, отвечающего за работу с совокупностью единиц обработки;
2. нижнего, отвечающего за обработку конкретной единицы генерации.

Верхний уровень содержит в себе один лишь этап - *Init*. Его первая задача состоит в организации пользовательского интерфейса. В рассматриваемом проекте таким интерфейсом выступает CLI, позволяющий задать один или несколько файлов-дескрипторов либо каталог, содержащий такие файлы. Далее по этим дескрипторам строятся единицы генерации, а также выполняется разрешение связей между словарями и секциями.

Нижний уровень объединяет обработку одной конкретной единицы генерации и является содержательно более сложным. Здесь обработка представлена как последовательность инкапсулированных этапов. Такая декомпозиция соответствует функциональному стилю проектирования, в котором система выражается как цепочка преобразований между типизированными представлениями данных [7; 8]. Каждый этап получает своё входное состояние, выполняет ограниченный набор преобразований и передаёт дальше уже более насыщенную семантикой структуру.

Для нижнего уровня выделены три этапа:

1. `Raw` — синтаксическое чтение входных форматов;
2. `Compile` — семантическая компиляция в прикладное промежуточное представление;
3. `Rust` — генерация итогового программного кода.

На рисунке 1 показана общая схема работы системы.

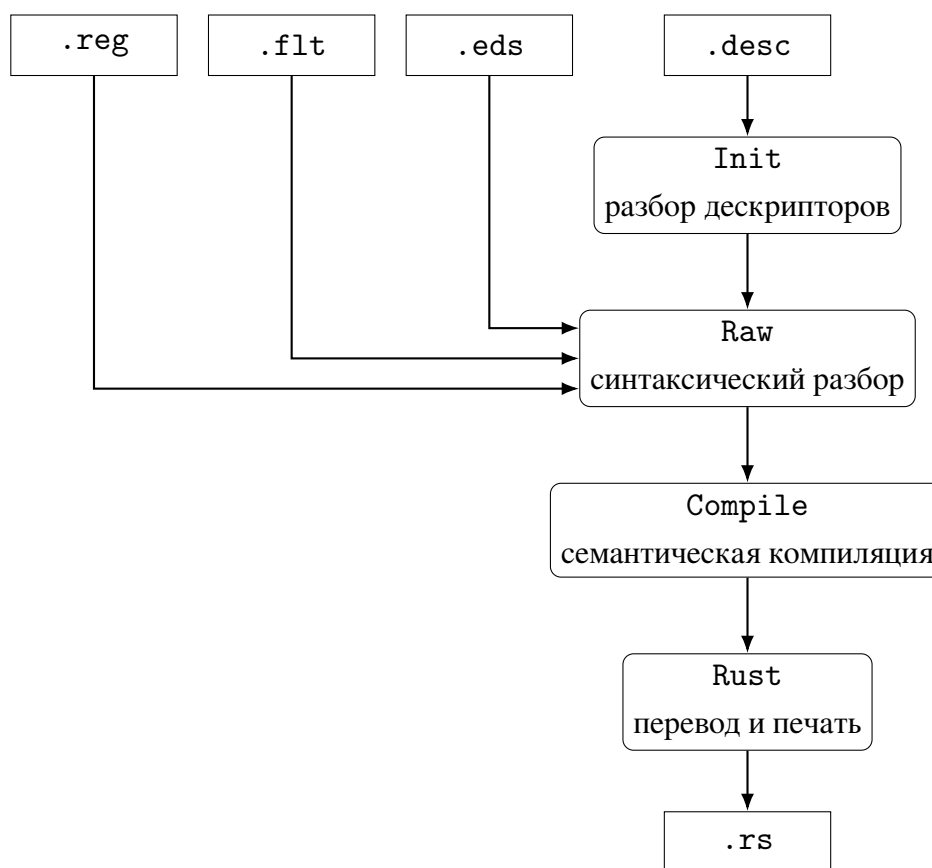


Рисунок 1 – Общий конвейер преобразования входных файлов в Rust-модули

2.3 Монадическая архитектура этапов проекта

Одним из ключевых архитектурных решений проекта является организация вычислений через набор вложенных монадических абстракций. Введём следующие типы, которые далее используются как базовые строительные блоки всех этапов:

Листинг 1 – Базовые типы монадического каркаса проекта

```
1 type Run = ExceptT GeneratorError IO
2 type EnvCtx r s a = ReaderT r (StateT s Run) a
```

Тип `Run` образует центральную вычислительную среду программы. Он сочетает стандартную монаду ввода/вывода `IO` с монадным трансформатором исключений `ExceptT`, отвечающим за корректное прерывание при критических ошибках [9; 10]. Уже на этом уровне задаётся важное свойство архитектуры: побочные эффекты и ошибки не рассеиваются по коду произвольным образом, а собираются в единый управляемый контекст. Это соответствует идиомам `mtl`, где эффекты описываются явно через ограничения и стек трансформеров [11].

Однако для проектирования отдельных этапов одного `Run` недостаточно. Этапы компиляции и генерации работают не только с эффектами ввода-вывода и ошибками, но и с двумя разными классами данных:

- неизменяемыми входными данными этапа;
- изменяемым рабочим состоянием, которое этот этап последовательно строит.

Именно поэтому поверх `Run` вводятся `ReaderT` и `StateT`. В архитектурном смысле это означает следующее.

`ReaderT` используется для данных, которые поступают на вход этапа и после этого не меняются. Это «контекст этапа»: то, что должно быть доступно многим функциям, но не должно быть случайно модифицировано. Такой выбор дисциплинирует код. Если структура данных помещена в `Reader`, то

любая функция этапа обязана относиться к ней как к источнику входных фактов, а не как к накопителю промежуточных результатов.

`StateT`, напротив, используется для данных, над которыми этап ведёт последовательную работу. Это уже не вход, а строящийся результат: карты объектов, множества импортов, частично построенные описания, накопленные преобразования. Помещение таких данных в `State` делает саму природу вычисления явной: этап постепенно обогащает своё состояние, не разрушая при этом неизменяемого входного контекста.

Присутствие `IO` в основании монадического стека связано с тем, что связь с внешним миром возникает не только при чтении и записи файлов. Она также проявляется в ряде служебных операций различных этапов, прежде всего в инфраструктуре обработки ошибок и взаимодействии с файловой системой. Поэтому `IO` в данном случае является не случайным добавлением, а частью базовой вычислительной среды.

Таким образом, архитектурная идея проекта может быть сформулирована так: каждый содержательный этап строится как преобразование *неизменяемого входа* в *накапливаемый результат* в контексте управляемых ошибок и `IO`. В терминах монадического стека это означает размещение входных данных в `ReaderT`, рабочего состояния в `StateT`, а ошибок и побочных эффектов в `ExceptT IO`.

Следует отметить, что не всякий этап обязан использовать полный стек целиком. Этапы `Init` и `Raw` в основном работают в `Run` и используют полиморфные ограничения `MonadError`, `MonadIO`, поскольку их преобразования сравнительно прямолинейны. Зато `Compile` и `Rust` используют полноценный `EnvCtx`, потому что именно в них наиболее отчётливо проявляется разделение между входным контекстом и накапливаемым рабочим состоянием.

3 Этап инициализации

3.1 Назначение инициализации

Этап `Init` занимает в архитектуре проекта особое место. Он ещё не работает ни с объектами словаря, ни с регистровыми интерпретациями, ни с Rust-кодом. Его задача другая: собрать и нормализовать конфигурацию запуска так, чтобы последующие этапы уже получили согласованное множество единиц генерации. Иными словами, `Init` отвечает за подготовку пространства задач, а не за содержательное преобразование самих данных словаря объектов.

3.2 Архитектура инициализации

На данном этапе отсутствует необходимость в накоплении сложного промежуточного состояния: его задача состоит не в поэтапной переработке данных, а в последовательной валидации и нормализации конфигурации. Поэтому весь этап естественным образом выполняется в монаде `Run`.

Функция верхнего уровня `initDesc` комбинирует три логически различные операции:

- чтение аргументов командной строки;
- разбор одного или нескольких файлов `desc`;
- разрешение зависимостей между `Dictionary` и `Section`.

Эта композиция особенно важна потому, что ошибки конфигурации должны быть обнаружены как можно раньше. Если разрешить переход к этапу `Raw` без предварительной проверки дескрипторов, то ошибки в `Uses`, дубликатах имён и типах модулей начнут проявляться гораздо позже, уже на стадии обработки содержательных данных.

Архитектура этапа схематически представлена на рисунке 2.

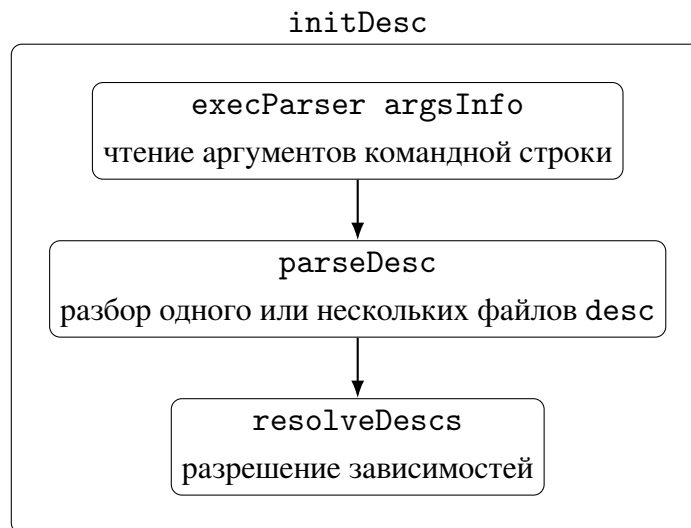


Рисунок 2 – Функциональная диаграмма инициализации

Функции имеют следующие сигнатуры:

Листинг 2 – Сигнатуры основных функций этапа Init

```

1 initDesc :: Run [BuildUnit]
2 execParser argsInfo :: IO InputArg
3 parseDesc :: (MonadIO m, MonadError InitError m)
4             => InputArg -> m [(String, Desc)]
5 resolveDescs :: MonadError ResolveError m => [(String, Desc)] -> m [BuildUnit]

```

3.3 Типы Desc и BuildUnit

Содержательная структура конфигурации задаётся типами Desc и BuildUnit. Ниже приведён их ключевой фрагмент:

Листинг 3 – Ключевые типы этапа инициализации

```

1 data DescType
2   = Dictionary
3   | Section

4 data Desc = Desc {
5     _descInput  :: String,
6     _descName   :: String,
7     _descType   :: DescType,
8     _descEDS    :: String,
9     _descFLT    :: Maybe String,

```

```
10     _descREG      :: Maybe String,
11     _descUses     :: [String],
12     _descOutput   :: String
13 }

14 data BuildUnit = BuildUnit
15   { buMain        :: Desc
16   , buSections    :: [Desc]
17   }
```

С архитектурной точки зрения здесь важно следующее. `Desc` описывает не весь проект целиком, а один модуль вывода. Это позволяет рассматривать генератор как систему, способную обрабатывать не только единственный словарь, но и множество связанных модулей. Тип `BuildUnit`, в свою очередь, переводит эту конфигурацию в форму, удобную для запуска дальнейших этапов: один основной словарь и набор подключённых секций.

Такой переход от `Desc` к `BuildUnit` устраняет двусмысленность в дальнейшей обработке. Начиная с этого момента все последующие этапы уже работают не с набором разрозненных файлов конфигурации, а с завершённой логической единицей генерации.

3.4 Разбор аргументов и выбор набора дескрипторов

Функция `parseDesc` поддерживает несколько режимов запуска: обработку всех дескрипторов из каталога `input`, обработку конкретного файла и обработку файлов из произвольного каталога.

Сам этап не хранит дополнительного состояния. Он не накапливает промежуточную карту дескрипторов во внешнем `State`, а выполняет трансформации последовательно, передавая результат от одной операции к другой. Это согласуется с тем, что `Init` в основном решает задачу валидации и нормализации конфигурации, а не многошаговой компиляции.

3.5 Парсинг формата `desc`

Файл `desc` представляет собой конфигурационное описание в формате «ключ = значение». Парсер `descParser` извлекает необходимые поля, проверяет наличие обязательных значений и преобразует строковое поле `Type` в

конструктор `DescType`.

Этот парсер не просто считывает набор строковых полей. Он сразу же накладывает на входные данные базовую структуру проекта: разделяет обязательные и необязательные поля, нормализует тип словаря, преобразует список зависимостей `Uses` и задаёт каталог вывода по умолчанию. Тем самым на последующих этапах уже не приходится повторно решать задачи низкоуровневой валидации конфигурации.

3.6 Разрешение зависимостей и проверка ограничений

Наиболее содержательная часть этапа `Init` сосредоточена в модуле `Init.Resolver`. Здесь выполняются четыре проверки:

- отсутствие повторяющихся имён дескрипторов;
- отсутствие ссылок на несуществующие модули;
- использование в `Uses` только модулей типа `Section`;
- отсутствие собственного списка `Uses` у секций.

После этого выполняется построение `BuildUnit`. Принципиально важно, что именно на этом шаге проект отделяет корневые модули от секций и тем самым завершает анализ зависимостей. Ключевая логика выглядит следующим образом:

Листинг 4 – Разрешение связей между `Dictionary` и `Section`

```
1 resolveDescs :: MonadError ResolveError m => [(String, Desc)] -> m [BuildUnit]
2 resolveDescs
3   =   dubsCheck
4   >=> pure . M.fromList
5   >=> misRefsCheck
6   >=> invUseCheck
7   >=> sectionUsesCheck
8   >=> resolve
```

Архитектурная ценность такого решения состоит в том, что весь последующий конвейер получает уже непротиворечивую конфигурацию. Это снижает связность между этапами: `Raw`, `Compile` и `Rust` не обязаны повторно

рассуждать о том, корректно ли устроены связи между словарями и секциями. Все подобные вопросы локализованы внутри `Init`.

3.7 Результат этапа инициализации

Итогом этапа является список `BuildUnit`, то есть завершённых единиц генерации, каждая из которых содержит один модуль `Dictionary` и список его секций. Такой результат можно рассматривать как границу между инфраструктурным и содержательным уровнями программы. До этого момента система решает задачу конфигурирования, а начиная со следующего этапа уже работает с самими входными данными предметной области.

4 Этап синтаксического разбора

4.1 Назначение синтаксического разбора

После завершения инициализации проект переходит к этапу Raw, задача которого состоит в чтении и синтаксическом разборе входных форматов и их приведении к первичным типизированным структурам. Важно подчеркнуть, что на этом этапе ещё не выполняется семантическая интерпретация словаря объектов. Этап Raw не решает, как именно следует понимать тот или иной объект; он лишь достоверно переносит содержимое файлов в внутренние структуры Haskell.

Такое разделение принципиально. Если бы синтаксический разбор и семантическая компиляция были смешаны, то проекту пришлось бы обрабатывать синтаксические ошибки, проблемы фильтрации и предметные правила интерпретации в одном и том же наборе функций. Текущая архитектура избегает этого: сначала строится сырое представление, а затем уже выполняется его нормализация и осмысление.

4.2 Архитектура синтаксического разбора

Задача этапа Raw состоит в чтении и синтаксическом разборе трёх входных форматов: EDS, а также FLT и REG, если они указаны в дескрипторе. На этом шаге проект ещё не пытается установить содержательную связь между полученными данными. Он лишь строит отображение текстовых файлов во внутренние представления, пригодные для последующей семантической обработки.

Именно поэтому архитектура этапа оказывается сравнительно простой. Все три входных файла читаются независимо друг от друга, поскольку на уровне синтаксического разбора между ними ещё не установлены предметные зависимости. Словарь объектов задаёт одну грамматику, фильтр — другую, регистровые интерпретации — третью. Следовательно, порядок их разбора несущественен, а сами операции могут быть выполнены параллельно. Такое решение уменьшает связанность функций этапа и подчёркивает, что общая семантическая сборка начинается лишь на следующем шаге конвейера.

ра.

С точки зрения внутренней организации здесь особенно важно, что этап сразу строит завершённую структуру `RawCtx`, не накапливая промежуточную карту результатов по мере вычисления. Он выполняет несколько независимых отображений из файла в сырое типизированное представление, после чего объединяет их в единый контекст следующего этапа. Поэтому его основные функции имеют следующий вид:

Листинг 5 – Этап Raw

```
1 runRaw :: Desc -> Run RawCtx
2 parseDictionary :: (MonadError GeneratorError m, MonadIO m) => Desc -> m RawObjs
3 parseFilter :: (MonadError GeneratorError m, MonadIO m) => Desc -> m RawFltObjs
4 parseRegister :: (MonadError GeneratorError m, MonadIO m) => Desc -> m RawRegObjs
```

Графически архитектура этапа представлена на рисунке 3.

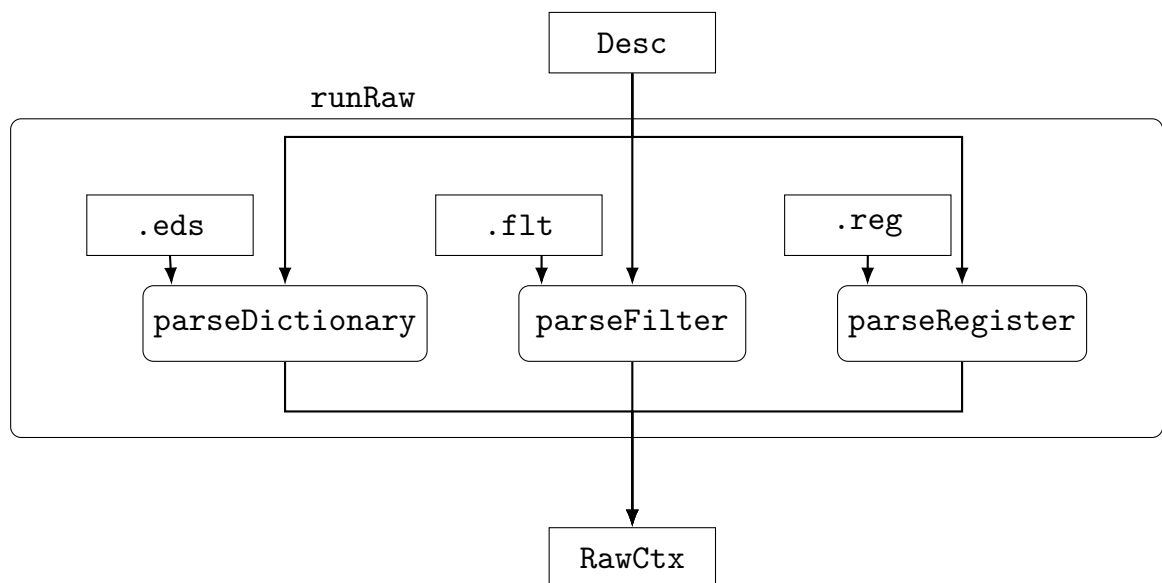


Рисунок 3 – Диаграмма синтаксического разбора

4.3 Границы этапа

Входом этапа служит значение типа `Desc`. Оно содержит сведения о том, где расположены файлы `.eds`, `.flt` и `.reg`, а также указание на то, какие из дополнительных файлов вообще заданы. Именно по этим путям и выполняется считывание входных данных.

Результат этапа Raw инкапсулируется в типе `RawCtx`:

```
1 data RawCtx = RawCtx
2   { _rawObjs :: RawObjs
3     , _rawFlts :: RawFltObjs
4     , _rawRegs :: RawRegObjs
5   }
```

С архитектурной точки зрения `RawCtx` играет ту же роль, которую `BuildUnit` играл для этапа инициализации: это завершённое входное представление следующего шага конвейера. В нём ещё нет декодированных типов данных `CANopen` и нет собранных прикладных объектов, но уже есть аккуратно разобранные структуры:

- карта сырого словаря объектов;
- список элементов фильтра;
- карта сырых регистровых описаний.

Именно `RawCtx` затем выступает входным контекстом этапа `Compile`. Это наглядно подтверждает ранее описанное архитектурное решение: результат одного этапа становится неизменяемой структурой входных данных для следующего.

4.4 Комбинаторные парсеры как абстрактная вычислительная модель

Прежде чем переходить к разбору конкретных форматов `EDS`, `FLT` и `REG`, необходимо кратко описать общую идею комбинаторных парсеров как математической конструкции. Такой парсер удобно понимать не как отдельную внешнюю утилиту и не как таблицу переходов, а как значение некоторого функционального типа, которое преобразует входную последовательность символов в результат разбора и остаток входа [12—14]. В самой простой форме эта идея выражается так:

```
1 type Parser a = Input -> [(a, Input)]
```

Такая запись важна именно с теоретической точки зрения. Она показывает, что парсер может рассматриваться как вычисление над языком входных слов, возвращающее значение типа `a` и новое состояние потока. Следовательно, парсер принадлежит к тому же классу объектов функционального мира, что и обычные функции, а значит, над ним можно строить композицию, абстракцию и более сложные операторы второго порядка.

Название «комбинаторный парсер» связано с тем, что сложный парсер строится не непосредственным описанием всей грамматики целиком, а композицией более простых парсеров при помощи комбинаторов. В теории комбинаторов комбинатором называют высокоуровневую функцию, которая принимает функции в качестве аргументов и строит из них новую функцию. В этом смысле парсер для выражения, записи или блока конфигурации можно получать как алгебраическое сочетание парсеров для атомарных компонентов.

Исторически такая картина тесно связана с идеями лямбда-исчисления. Лямбда-исчисление рассматривает вычисление как последовательность преобразований выражений посредством абстракции и применения. Комбинаторный подход к парсингу наследует эту идею: разбор текста выражается через композицию чистых преобразований, а логика грамматики задаётся не внешним механизмом, а непосредственно в языке программирования. Поэтому комбинаторные парсеры естественно вписываются в функциональную парадигму, где функции являются полноправными значениями и могут быть свободно комбинированы.

С алгебраической точки зрения набор парсеров образует структуру, над которой можно выделить несколько характерных операций. Во-первых, существует операция последовательной композиции: если один парсер успешно разбирает префикс входа, другой может продолжить разбор оставшейся части. Во-вторых, существует операция альтернативы: если одна ветвь разбора не подошла, можно попытаться применить другую. В-третьих, существуют операции подъёма обычных значений и функций в пространство парсеров.

Именно из таких операций в функциональных библиотеках формируются экземпляры `Functor`, `Applicative`, `Alternative` и `Monad` [15].

В терминах `Functor` парсер допускает отображение уже разобранного значения без изменения самой логики чтения входа. Это соответствует ситуации, когда текстовая форма уже распознана, но результат нужно преобразовать в более удобный внутренний тип. В терминах `Applicative` парсеры образуют более богатую композиционную алгебру: один парсер может построить функцию, а другой — аргумент для неё. Такой стиль удобен, когда структура входа заранее известна и разбор можно представить как построение значения по фиксированному шаблону.

Монадическая структура идёт дальше. Она позволяет строить следующий шаг разбора в зависимости от результата предыдущего. С математической точки зрения это означает переход от простой композиции к зависимому вычислению, где последующая грамматика может определяться уже полученным значением. Именно это свойство делает комбинаторные парсеры особенно выразительными для вложенных и контекстно-чувствительных структур, где порядок применения правил нельзя полностью зафиксировать заранее.

Ещё одно важное свойство комбинаторных парсеров связано со *свободным* построением грамматики из элементарных парсеров и комбинаторов. Иначе говоря, сложная грамматика возникает как выражение в сигнатуре допустимых операций без введения дополнительного внешнего формализма. Описание синтаксиса записывается как программа, чья форма сама по себе отражает структуру языка. Это отличает комбинаторный подход от многих традиционных систем генерации парсеров, где грамматика задаётся во внешнем формализме и лишь затем компилируется в код.

С практической точки зрения важным следствием этой алгебраической природы является локальность. Каждый небольшой парсер описывает ограниченный фрагмент языка, а затем включается в более крупную конструкцию. Благодаря этому доказательство корректности, отладка и расширение грамматики выполняются инкрементально: достаточно понять свойства отдельных комбинаторов и правила их сочетания.

Для функциональных языков такая модель особенно естественна. Если парсер — это значение, а не внешний артефакт, то его можно хранить в

переменной, передавать в функцию, комбинировать с другими парсерами и параметризовать. Поэтому выбор `Parsec` в рассматриваемом проекте связан не только с удобством библиотеки, но и с общей математической совместимостью между функциональной природой языка `Haskell` и комбинаторной природой самого механизма синтаксического анализа.

4.5 Тип `Ind`

Модуль `Raw.Common` содержит общее описание лексики и базовые синтаксические комбинаторы для всех парсеров. Наиболее важным типом здесь является `Ind`, описывающий индекс объекта:

Листинг 8 – Тип индекса словаря объектов

```
1 data Ind
2   = Index Int
3   | Subindex Int Int
4   deriving (Eq, Ord)
```

Выделение подындкса в отдельный конструктор является принципиальным проектным решением. Это означает, что основным идентификатором элемента в объектном словаре является не просто его индексом в целочисленном значении, а именно отдельным типом индекса, который, как и в самом объектном словаре, может обозначать как полный индекс, так и индекс с подиндексом. Благодаря этому вплоть до этапа перевода в языковую модель `Rust` подобъекты остаются равноправными с объектами. Такое решение упрощает фильтрацию, склейку и доступ как к исходному значению `OD`, так и к интерпретации, заданной пользователем при помощи регистровых карт.

4.6 Парсинг словаря объектов EDS

Модуль `Raw.Eds` разбирает словарь объектов как последовательность блоков, каждый из которых состоит из заголовка и множества пар «ключ – значение». Результат представляется типом `RawObjs = Map Ind ObjBody`. В таком виде словарь всё ещё не интерпретирован семантически, но уже имеет точную адресацию и структуру.

С точки зрения проектирования такое решение оправдано следующим образом. На этапе Raw нет необходимости сразу же декодировать типы данных или различать массивы и записи. Гораздо важнее гарантировать, что каждая запись словаря привязана к правильному индексу и что дублирование индексов будет обнаружено немедленно. Соответствующая логика реализована функцией `insertObj`, запрещающей повторную вставку объекта с тем же ключом.

4.7 Парсинг фильтра FLT

Формат FLT устроен значительно проще. Он задаёт набор индексов, помеченных как элементы белого или чёрного списка. Однако даже при такой простоте выбор отдельного парсера является оправданным: фильтр обладает собственной семантикой, а его синтаксис нужно поддерживать и валидировать независимо от словаря объектов.

Внутреннее представление фильтра обладает следующими свойствами:

- сохраняет знак включения или исключения;
- опирается на тот же тип `Ind`, что и словарь объектов;
- сохраняет информацию в последовательности, пригодной для последующей нормализации.

Таким образом, даже предельно компактный формат включён в общий архитектурный стиль проекта: сначала синтаксический разбор, затем семантическая нормализация.

Смысл такого проектного решения состоит в том, что фильтр не должен дублировать возможности словаря объектов. Его задача заключается лишь в задании области генерации. Чем проще синтаксис фильтра, тем меньше риск смешать в одном формате вопросы адресации, структуры объекта и прикладной интерпретации.

4.8 Парсинг регистровых интерпретаций REG

Наиболее сложным парсером этапа Raw является Raw.Reg. Это связано с тем, что файл REG описывает не просто набор значений, а фактически язык

дополнительной интерпретации объектов словаря. В нём можно задавать:

- переименование объекта;
- список атрибутов;
- представление объекта как `enum`;
- представление объекта как `struct`;
- диапазоны битов и вложенные перечисления для полей структуры.

Синтаксически запись REG может содержать имя, список атрибутов и тело описания. Само тело, в свою очередь, может задавать либо перечисление, либо структуру. Для структур допускается указание отдельных битов, диапазонов битов и локальных перечислений для отдельных полей. Этого уже достаточно, чтобы на следующем этапе перейти от сырого словарного значения к прикладному типу с более богатой внутренней структурой.

Ключевые типы этого мини-языка выглядят следующим образом:

Листинг 9 – Основные типы парсера REG

```
1 data FieldBits
2   = One Int
3   | Range (Int, Int)

4 data StructField = StructField
5   { _sfBits :: FieldBits
6     , _sfName :: FieldName
7     , _sfType :: Maybe FieldType
8     , _sfEnum :: Maybe EnumCorpus
9   }

10 data RawRegister a = RawRegister
11   { _rawName :: Maybe String
12     , _rawAttr :: Maybe [String]
13     , _rawBody :: Maybe a
14   }
```

Парсер REG строит не конечную предметную модель, а сырую структуру, пригодную для дальнейшего согласования со словарём объектов. Именно

по этой причине итоговое семантическое соединение EDS и REG выполняется только на этапе `Compile`.

4.9 Результат этапа Raw

Итогом этапа является `RawCtx`, содержащий уже разобранные, но ещё не интерпретированные данные. Это очень важная архитектурная граница. До неё проект занимается лишь корректным переносом текстовых форматов во внутренние структуры. После неё система начинает рассматривать эти структуры как предметные сущности и строить на их основе прикладную модель кодогенерации.

Именно поэтому этап Raw удобно характеризовать как *синтаксический контур безопасности* проекта. Его задача не в том, чтобы сделать данные полезными для итогового Rust-кода, а в том, чтобы гарантировать: к семантической компиляции попадут только такие входы, которые уже согласованы с синтаксисом соответствующих форматов. Вся последующая архитектура конвейера строится на этом предположении.

5 Компиляция промежуточного представления

5.1 Назначение компиляции промежуточного представления

Этап `Compile` является центральным звеном всей системы. Если `Init` решает задачу конфигурирования, а `Raw` отвечает за синтаксическую достоверность, то `Compile` выполняет семантическую сборку предметной модели. Именно здесь словарь объектов, фильтр и регистровые описания начинают рассматриваться как части единой системы знаний об устройстве.

Содержательный смысл этого этапа состоит в переходе от синтаксически корректных, но ещё разрозненных представлений к единой промежуточной модели, пригодной для дальнейшего отображения в `Rust`. На входе `Compile` получает три независимых класса данных: сырые записи словаря объектов, записи фильтра и сырые регистровые интерпретации. На выходе же формируется отображение индексов в законченные прикладные объекты, в которых уже согласованы адресация `CANopen`, область генерации и дополнительная интерпретация значений.

Именно поэтому этап `Compile` нельзя свести ни к простому декодированию, ни к простому объединению структур. Он решает сразу несколько задач различной природы:

- нормализует фильтр и тем самым строит точную область дальнейших вычислений;
- декодирует словарь объектов из текстовой формы в типизированное представление;
- декодирует регистровые описания в прикладную модель интерпретации значений;
- согласует обе модели по индексам;
- сворачивает плоскую индексную структуру в переменные, массивы и записи.

В результате именно `Compile` выступает границей между двумя различными способами представления данных. До него проект работает с фай-

лами и их синтаксисом. После него система оперирует уже не текстовыми конструкциями, а предметными сущностями, которые обладают устойчивой типовой структурой и потому могут служить основанием для генерации программного интерфейса.

5.2 Архитектура компиляции промежуточного представления

Архитектурно этот этап особенно интересен тем, что он первым явно разводит два различных класса данных: входной контекст `RawCtx` и рабочее состояние `CompileState`. Благодаря этому сырые результаты синтаксического разбора сохраняются как неизменяемое основание вычислений, а все промежуточные результаты компиляции собираются в отдельной структуре состояния.

Внутреннее состояние этапа задаётся следующим образом:

Листинг 10 – Состояние этапа `Compile`

```
1 data CompileState = CompileState
2   { _idxFilter :: Set Ind
3     , _objDict   :: Map Ind ObjectDictionaryEntry
4     , _regDict   :: Map Ind RegisterEntry
5     , _grouped   :: Map Ind CompleteObject
6   }
7
7 type CompileM a = EnvCtx RawCtx CompileState a
```

Каждое поле `CompileState` соответствует отдельной фазе работы:

- `idxFilter` хранит уже нормализованный набор индексов;
- `objDict` содержит декодированные записи словаря объектов;
- `regDict` содержит декодированные регистровые описания;
- `grouped` содержит окончательно собранные объекты прикладного уровня.

Именно поэтому `CompileState` удобно понимать не просто как память этапа, а как формализованную историю его вычислений. Сначала в нём появ-

ляется фильтр, затем карты словаря и регистров, затем итоговая структура завершённых объектов.

Общая последовательность операций в `Compile` выражена компактно:

Листинг 11 – Основной поток этапа `Compile`

```
1 runCompileM :: CompileM ()
2 runCompileM = do
3   compileFilter >=> expandFlt
4   compileObjectDictionary
5   compileRegisterMaps
6   glueObjReg >=> foldCompiled
```

Эта запись важна сама по себе. Она показывает, что этап компиляции не является набором разрозненных вспомогательных функций. Напротив, внутри него существует ясный и последовательный конвейер: нормализация области видимости объектов, декодирование словаря, декодирование регистровых описаний, склейка двух моделей и финальная свёртка.

5.3 Нормализация фильтра

Первой задачей этапа становится преобразование сырого списка элементов фильтра в нормализованное множество индексов. Здесь возникает естественный вопрос: почему нельзя сразу использовать пользовательский ввод в исходном виде?

Причина состоит в том, что подобъекты в словаре не являются полностью самостоятельными сущностями. Во многих случаях верхний индекс задаёт некоторый составной объект, а подындексы описывают его внутренние компоненты. Следовательно, между верхнеуровневой и дочерней адресацией существует структурная зависимость, которую невозможно корректно учесть простым копированием пользовательского списка.

Наличие отдельной стадии нормализации позволяет упростить остальные части системы. Без неё пришлось бы либо преждевременно связывать индекс и подындекс в более жёсткую структуру, либо усложнять все последующие алгоритмы учётом специальных случаев. В частности, трудно было

бы выразить ситуации, когда пользователь выбирает запись целиком, но исключает отдельное поле через чёрный список, либо, напротив, явно адресует только часть составного объекта.

Модуль `Compile.Filter` сначала проверяет, что FLT не смешивает белый и чёрный список, затем преобразует исходные записи в единое множество и только после этого выполняет нормализацию относительно универсума индексов из словаря объектов.

Особую роль играет функция `addSubIndexes`, автоматически добавляющая подиндексы для выбранных верхнеуровневых индексов. Это важное проектное решение: пользователь мыслит словарём объектов на уровне сущностей, тогда как внутренний формат словаря содержит и верхние, и дочерние элементы. Нормализация фильтра снимает этот разрыв между человеческим представлением и внутренней структурой данных.

В результате фильтр из локальной синтаксической конструкции превращается в точный механизм управления областью дальнейших вычислений. Благодаря нормализации, опирающейся на универсум всех объектов, определённых в OD, проект получает единое множество индексов, подлежащих обработке, независимо от того, использовал ли пользователь белый или чёрный список. После этого все последующие операции этапа `Compile` работают уже не со всем исходным словарём, а только с тем его подмножеством, которое было сознательно допущено к генерации.

5.4 Декодирование словаря объектов

После того как множество интересующих индексов определено, этап компиляции переходит к декодированию словаря объектов, при этом компилируются только те объекты, которые ранее были допущены фильтром. Здесь сырые записи `Map Ind ObjBody` превращаются в элементы типа `ObjectDictionaryEntry`. Ключевые типы этого слоя приведены ниже:

Листинг 12 – Ключевые типы словаря объектов после декодирования

```
1 data DataType =  
2     UNSIGNED8 | UNSIGNED16 | UNSIGNED32 | UNSIGNED64  
3     | INTEGER8   | INTEGER16   | INTEGER32   | INTEGER64
```

```

4   | BOOLEAN      | REAL32
5   | VISIBLE_STRING | OCTET_STRING | UNICODE_STRING
6   | TIME_OF_DAY | TIME_DIFFERENCE | DOMAIN

7 data ObjectType = VariableType | ArrayType | RecordType

8 data ObjectDictionaryEntry =
9     VariableEntry BasicEntry
10    | ArrayEntry BasicDesc
11    | RecordEntry BasicDesc

```

В ходе декодирования модуль `Compile.Object` извлекает тип данных, тип записи, режим доступа и базовое описание объекта. Таким образом, текстовый словарь объектов впервые преобразуется в собственную типизированную модель проекта.

Важно, что функция `decodeObject` принципиально сохраняет плоскую форму словаря: массивы и записи пока ещё не сворачиваются в законченные структуры. На данном шаге система только декодирует отдельные элементы и сопоставляет им предметную семантику `CANopen`, но не пытается преждевременно группировать подындексы.

Такой подход упрощает дальнейшую интеграцию с `REG`. Пока записи остаются плоскими, их легко сопоставлять по индексу с регистровыми интерпретациями.

5.5 Декодирование регистровых описаний

Следующим шагом выступает декодирование `REG`-описаний в тип `RegisterEntry`. На этом этапе уже недостаточно чисто синтаксической структуры из `Raw.Reg`; необходимо превратить её в прикладную модель, которую можно будет совмещать со словарём объектов.

Ключевые типы этого слоя имеют вид:

Листинг 13 – Ключевые типы регистровой интерпретации

```

1 data RegisterField
2   = FBool Int
3   | FEnum (Int, Int) [(String, Int)]

```

```

4   | FInt (Int, Int)

5   data Register
6     = RStruct [(String, RegisterField)]
7     | REnum [(String, Int)]

8   data RegisterEntry = RegisterEntry
9     { _reName  :: Maybe String
10      , _reAttrs :: Maybe [Attribute]
11      , _reBody  :: Maybe Register
12      }

```

Формально декодирование устроено сверху вниз. Значение типа `RegisterEntry` задаёт одну регистровую интерпретацию и состоит из трёх опциональных частей: *имени*, *атрибутов* и *тела*. Отсутствие тела означает, что запись может лишь переименовывать объект или снабжать его атрибутами, не меняя внутреннюю структуру значения.

Если тело присутствует, оно имеет тип `Register`. Здесь возможны два случая. Конструктор `REnum` описывает объект как перечисление, то есть как список пар «имя варианта – числовое значение». Конструктор `RStruct` описывает объект как структуру, то есть как список именованных полей. Каждое поле структуры имеет тип `RegisterField`, а значит, может обозначать одиночный бит (`FBool`), диапазон битов, трактуемый как целое число (`FInt`), либо диапазон битов, трактуемый как локальное перечисление (`FEnum`).

Такое устройство типов задаёт строгую дисциплину интерпретации. Во-первых, проект явно различает интерпретацию всего значения как перечисления и интерпретацию значения как составной структуры полей. Во-вторых, для структур запрещается произвольное смешение форм поля: одиночный бит, диапазон числовых битов и диапазон с локальным перечислением принадлежат разным вариантам одного типа. Следовательно, после декодирования регистровая интерпретация уже не является свободной текстовой записью, а становится элементом строгой алгебры вариантов, пригодной для последующего отображения в типы Rust.

Архитектурно здесь важно, что регистровое описание декодируется отдельно и накапливается в собственной карте `regDict`. Это позволяет на сле-

дующем шаге согласовывать его со словарём объектов по индексу, не смешивая обе линии обработки раньше времени.

Важно подчеркнуть, что этот слой также содержит собственную дисциплину ошибок. Если поле перечисления задано в неверном формате, если тип поля противоречит диапазону битов или если указан неизвестный атрибут, проект генерирует ошибку именно на уровне `Compile.Register`. Следовательно, ответственность за корректность прикладной интерпретации сосредоточена в том модуле, который непосредственно знает правила этой интерпретации.

5.6 Склейка словаря и регистровой карты

Когда и словарь объектов, и регистровая карта уже декодированы, проект переходит к их сопоставлению по индексу. Для этого используется тип `UnifiedEntry`:

Листинг 14 – Тип объединённой записи

```
1 data UnifiedEntry
2   = Native ObjectDictionaryEntry
3   | Modified ObjectDictionaryEntry RegisterEntry
```

Вариант `Native` означает, что объект сохраняет только сведения из словаря объектов. Вариант `Modified` означает, что протокольное описание объекта дополнительно снабжено прикладной регистровой интерпретацией. С точки зрения предметной области это крайне удачная модель: она выражает тот факт, что объект `CANopen` может существовать сам по себе, но при необходимости получать более богатую интерпретацию.

Именно здесь становится особенно заметно, почему словарь объектов не сворачивается в итоговые структуры сразу после чтения. До тех пор пока не выполнена склейка с `REG`, проект ещё не знает, какое именно прикладное представление следует использовать для конкретного объекта. Преждевременная свёртка только усложнила бы логику сопоставления и увеличила бы количество специальных случаев.

С практической точки зрения склейка выполняет ещё одну важную

функцию: она сохраняет обе точки зрения на один и тот же объект. В `Modified` остаются и исходная запись словаря, и регистровое описание. Благодаря этому последующие этапы могут использовать информацию из обеих моделей одновременно: например, имя и тип из словаря объектов и структуру полей из `REG`.

5.7 Свёртка в законченные объекты

После склейки этап `Compile.Folder` объединяет верхнеуровневые индексы и подындексы в законченные объекты типа `CompleteObject`:

Листинг 15 – Тип законченного объекта прикладного уровня

```
1 data CompleteObject
2   = CompVar    UnifiedEntry
3   | CompArray  ObjectDictionaryEntry Int UnifiedEntry
4   | CompStruct UnifiedEntry (IntMap UnifiedEntry)
```

Архитектурный смысл свёртки состоит в переходе от *плоской индексной модели* к *структурной модели* прикладного объекта. До этой точки проект работает с отдельными индексами и подындексами как с независимыми сущностями. После неё он может рассуждать об объекте как о переменной, массиве или структуре.

Это особенно важно для генерации Rust-кода. Язык вывода ориентирован на структурные типы, а не на плоские таблицы записей. Следовательно, без этапа свёртки проект не смог бы естественным образом отобразить словарь объектов в типы, поля и зависимости между ними.

Модуль `Compile.Folder` дополнительно фиксирует и ошибки предметного уровня: пустой массив, пустую структуру, попытку модифицировать саму структуру через `REG`, несогласованность между массивом и описанием его элемента. Это важно, потому что такие ошибки уже не относятся к синтаксису входных форматов. Они выражают нарушение логики предметной модели и потому закономерно локализованы именно на этапе семантической компиляции.

5.8 Результат этапа `Compile`

Результатом этапа является карта свёрнутых объектов `Compiled`. Её тип представлен в листинге 16.

Листинг 16 – Результат этапа `Compile`

```
1 type Compiled = Map Ind CompleteObject
```

Однако смысл этого результата не исчерпывается самой сигнатурой. Тип `Compiled` фиксирует, что к концу этапа проект окончательно покидает уровень разрозненных сырых записей и переходит к уровню завершённых прикладных объектов. Ключами отображения остаются индексы `Ind`, благодаря чему сохраняется связь с адресацией `CANopen`, но значения уже представлены как `CompleteObject`, то есть как переменные, массивы или структуры с учётом согласованной регистровой интерпретации.

Тем самым `Compiled` объединяет в одной модели несколько ранее раздельных аспектов:

- область генерации, заданную нормализованным фильтром;
- протокольную информацию из словаря объектов;
- прикладную интерпретацию, поступившую из REG;
- структурные связи между верхними индексами и подындексами.

Именно эта совокупность свойств делает результат этапа достаточным для дальнейшего перевода в Rust. Следующий этап уже не должен повторно решать, какие объекты следует генерировать, какому типу словарной записи они соответствуют, имеется ли у них дополнительная интерпретация и следует ли рассматривать их как простую переменную, массив или структуру. Все эти решения уже приняты на этапе `Compile` и выражены на уровне типов.

Кроме того, `Compiled` является результатом, из которого исключены те элементы, что не прошли фильтрацию или оказались несогласованными на уровне предметной модели. Иными словами, карта результата содержит только те объекты, которые одновременно принадлежат области генерации и

допускают корректную семантическую интерпретацию в рамках архитектуры проекта.

В этом смысле `Compiled` можно рассматривать как нормальную форму внутреннего представления словаря. Она ещё не зависит от языка вывода, но уже достаточно богата, чтобы служить универсальным основанием для отображения в конкретную программную модель. Именно поэтому этап `Rust` работает уже не с `RawCtx` и не с частными картами объектов и регистров, а непосредственно с `Compiled`.

Технически данная карта извлекается из поля `grouped` состояния `CompileState`. Однако архитектурно важно не само извлечение, а то, что `grouped` представляет собой итог всей последовательности вычислений этапа: нормализации, декодирования, склейки и свёртки. Следовательно, `Compiled` есть не просто очередная структура данных, а инвариантный результат всей семантической компиляции.

6 Перевод в Rust и генерация выходного кода

6.1 Назначение перевода в Rust

Назначение этапа Rust состоит в отображении результата семантической компиляции в готовый модуль языка Rust. Однако его задача шире, чем простая печать уже имеющихся структур в текстовом виде. На входе этап получает карту `Compiled`, то есть завершённое промежуточное представление словаря, а на выходе строит согласованное описание Rust-модуля, включающее типы объектов, дополнительные `impl`-блоки, аннотирующие типы `Index`, `Value`, `Dict`, систему импортов и итоговый текст файла.

Тем самым данный этап завершает последнее существенное отображение во всём конвейере: переход от внутренней предметной модели проекта к конструкции конкретного целевого языка. Если этап `Compile` отвечает на вопрос, *что именно* представляют собой объекты словаря, то этап Rust отвечает на вопрос, *в каком виде* эти объекты должны быть выражены в системе типов и модульной организации Rust.

С архитектурной точки зрения важно и то, что данный этап работает не только с одной картой объектов, но и с контекстом связей между модулями. Для `Dictionary` недостаточно перевести лишь собственные объекты: необходимо учитывать подключённые `Section`, формировать импорты между модулями и, при необходимости, строить общие аннотирующие типы на основе совокупности нескольких артефактов. Поэтому этап Rust одновременно решает задачу перевода, задачу сборки модульного интерфейса и задачу материализации результата в файловой системе.

Именно поэтому данный этап выделен в отдельный крупный архитектурный блок. Всё, что относится к отображению внутренней модели на язык Rust, сосредоточено в одном месте: двухуровневый перевод объектов, генерация аннотирующих типов, управление импортами и окончательный вывод модуля. Такое решение облегчает сопровождение текущего выходного языка и одновременно оставляет возможность в дальнейшем дополнять систему другими целевыми языками, не изменяя предыдущие этапы конвейера.

6.2 Архитектура перевода в Rust

Архитектура этапа Rust организована как последовательность операций над окружением переводчика `RustEnv` и накапливаемым состоянием `RustState`. Если на предыдущем этапе результатом служило отображение индексов в объекты `Compiled`, то здесь этого уже недостаточно. Для корректного формирования модуля необходимо учитывать не только перевод собственных объектов, но и связи между модулями, условную генерацию аннотирующих типов и окончательную сборку текстового представления файла.

Именно по этой причине этап не сводится к одной функции перевода. Его архитектура включает отдельные шаги, каждый из которых отвечает за собственный аспект итогового модуля. Сначала выполняется перевод внутренних объектов в Rust-представление, затем при необходимости строятся аннотирующие типы, после этого вычисляется система импортов и лишь на завершающем шаге происходит печать и запись результата в файл. Вся архитектура представлена на рисунке 4.

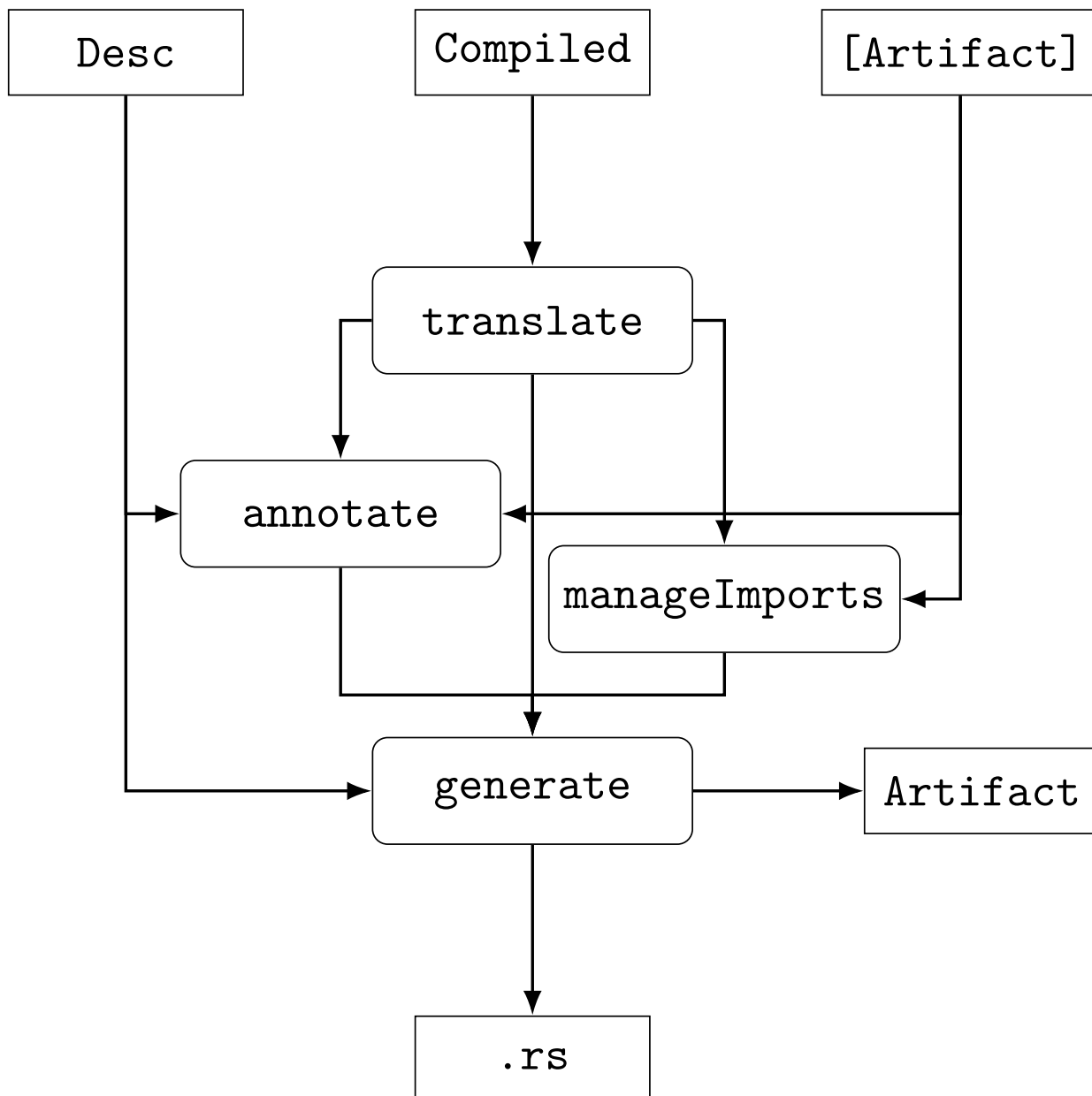


Рисунок 4 – Диаграмма перевода в Rust

Особое место в архитектуре этапа занимает тип `Artifact`. Его необходимость определяется тем, что перевод `Dictionary`-модуля зависит не только от его собственной карты объектов, но и от уже обработанных секций. Если бы функция верхнего уровня лишь записывала файл на диск и ничего не возвращала, информация о результатах перевода `Section` была бы потеряна для последующих вычислений. Между тем для `Dictionary` требуется знать не только имена секций для генерации импортов, но и результаты их внутреннего перевода, поскольку аннотирующие типы `Index`, `Value` и `Dict` строятся по объединённому контексту объектов текущего словаря и подключённых секций.

Тип `Artifact` решает эту задачу, фиксируя результат перевода модуля как устойчивую единицу представления. Он содержит три компонента: исходный дескриптор `Desc`, исходную карту `Compiled` и итоговое состояние `RustState`. Тем самым `Artifact` инкапсулирует как исходный контекст модуля, так и результат его перевода. Это позволяет использовать уже сгенерированные секции не только как файлы на диске, но и как структурированные данные, доступные последующим вызовам `runRust`.

Сигнатуры основных функций представлены в листинге 17.

Листинг 17 – Сигнатуры основных функций этапа Rust

```
1 runRust :: Desc -> [Artifact] -> Compiled -> Run Artifact
2 translate :: (MonadReader RustEnv m, MonadState RustState m, MonadIO m)
3             => m ()
4 annotate :: (MonadReader RustEnv m, MonadState RustState m, MonadIO m)
5             => m ()
6 manageImports :: (MonadReader RustEnv m, MonadState RustState m, MonadIO m)
7                 => m ()
8 generate :: (MonadReader RustEnv m, MonadState RustState m, MonadIO m)
9             => m ()
```

Функция `runRust` является внешней точкой входа этапа. Она получает дескриптор текущего модуля, список уже построенных артефактов секций и карту объектов `Compiled`. На основе этих данных формируется окружение `RustEnv`, затем выполняется внутренний процесс `runRustM`, а его итоговое состояние упаковывается в `Artifact`. Следовательно, `runRust` не только иницирует перевод, но и фиксирует его результат в форме, пригодной для дальнейшего переиспользования.

Функция `translate` отвечает за непосредственный перевод объектов из `CompleteObject` в `RustDesc`. Она применяет двухуровневый переводчик ко всем элементам текущей карты, накапливает успешные результаты в поле `descs` и формирует диагностические сообщения об ошибках перевода. Этот шаг задаёт основное содержательное наполнение будущего модуля: именно здесь возникает внутренняя модель Rust-типов.

Функция `annotate` выполняет условную генерацию аннотирующих типов и связанных с ними дополнительных `impl`-блоков. Она используется

только для модулей типа `Dictionary`. В отличие от `translate`, эта функция работает уже не только с собственными объектами текущего модуля, но и с артефактами секций. Благодаря этому аннотирующие типы строятся по расширенному контексту, отражающему всю совокупность объектов словаря.

Функция `manageImports` отвечает за вычисление замыкания импортов, необходимых для итогового файла. Она объединяет обязательные импорты, условные импорты, вытекающие из уже построенных Rust-описаний, и импорты секций. Тем самым данный шаг завершает сборку модульного интерфейса на уровне зависимостей между Rust-определениями.

Функция `generate` выполняет заключительную материализацию результата. Она извлекает из состояния переведённые описания, дополнительные `impl`-блоки, аннотирующие типы и список импортов, затем передаёт их в подсистему печати и записывает сформированный текст в выходной файл. Именно на этом шаге внутренняя модель этапа Rust превращается в внешний артефакт — компилируемый модуль языка Rust.

Связи между перечисленными функциями образуют строгую последовательность. `runRust` строит окружение и запускает процесс. `translate` порождает базовые описания объектов. `annotate`, опираясь на них и на артефакты секций, дополняет модель аннотирующими типами и внешними `impl`-блоками. `manageImports` завершает сборку зависимостей. Наконец, `generate` использует все накопленные компоненты для печати файла и формирования `Artifact`. Следовательно, каждая из этих функций не дублирует другую, а реализует отдельную фазу единого отображения из `Compiled` в итоговый Rust-модуль.

6.3 Модель языка Rust

Разработанная модель по своему объёму заслуживает отдельной работы. Кратко обозрим её здесь.

Для генерации корректного выходного кода недостаточно оперировать строками и шаблонами. Синтаксис Rust достаточно сложен: в нём одновременно присутствуют пути, `generic`-параметры, `generic`-аргументы, ссылки с временами жизни, ассоциированные типы, составные паттерны, блоки выра-

жений, методы с параметрами типов и другие конструкции, которые связаны между собой не линейно, а через вложенные и взаимно вложенные отношения [16]. Поэтому в проекте используется не текстовая, а структурная модель языка, то есть внутреннее абстрактное синтаксическое дерево.

При этом речь не идёт о полном воспроизведении официального синтаксиса Rust. В проекте моделируется только тот фрагмент языка, который необходим для решения поставленной задачи: описания типов, путей, выражений, паттернов, `impl`-блоков, импортов и других конструкций, реально используемых при генерации модулей словаря объектов.

При этом модель строится по рекурсивному принципу. Её базовые типы описывают не готовые объекты уровня модуля, а элементарные синтаксические сущности, из которых затем составляются более крупные конструкции. Важнейшую роль здесь играют типы `RustType`, `RustPath`, `Expr`, `Pattern`, а также сопутствующие типы `generic`-параметров, литералов, блоков и импортов.

Центральным типом для описания системы типов Rust выступает `RustType`. Его структура уже сама по себе является рекурсивной. Тип может быть примитивным (`TyPrim`), именованным (`TyNamed`), заданным через путь (`TyPath`), ссылочным (`TyRef`), параметризованным (`TyApp`), массивом (`TyArray`), кортежем (`TyTuple`), ассоциированным типом (`TyAssoc`) или срезом (`TySlice`). Почти все эти варианты вновь содержат внутри себя значения типа `RustType`. Поэтому типовая модель образует рекурсивное дерево, отражающее вложенную природу реального синтаксиса Rust.

Не менее важен тип `RustPath`, описывающий путь вида `crate::module::Type` или более сложные конструкции с параметрами. Путь состоит из списка сегментов `PathSeg`. Каждый сегмент включает идентификатор и аргументы. Аргументы, в свою очередь, могут быть отсутствующими, угловыми (`AngleArg`) или функциональными (`ParenthesizedArgs`). Здесь проявляется взаимная рекурсия модели: путь содержит аргументы, аргументы содержат `RustType`, а сами типы в варианте `TyPath` снова содержат путь.

`Generic`-механизм также встроен в общую рекурсивную структуру модели. Типы `GenericArg`, `GenericArgs`, `Generics`, `GenericParam` и

TraitBound позволяют выражать как фактические аргументы обобщённых типов, так и их формальные параметры и ограничения. Здесь снова возникает взаимная связность: TraitBound содержит RustPath и список RustType, RustType может включать GenericArgs, а сегменты пути тоже способны нести generic-аргументы. Тем самым модель не разбивает generics на внешний довесок, а встраивает их в самую синтаксическую ткань языка.

Рекурсивная природа особенно отчётливо проявляется на уровне выражений. Тип Expr описывает переменные, пути, литералы, вызовы, методы, обращения к полям, блоки, сопоставление с образцом, кортежи, структурные литералы, конструкторы перечислений, приведение типов, индексацию, унарные и бинарные операции, ссылки, присваивания, диапазоны и массивы. Практически каждый из этих вариантов содержит другие выражения, а некоторые из них одновременно содержат и типы, и пути. Например, ECall включает вызываемое выражение и список аргументов-выражений, EMethod содержит выражение-получатель, список RustType и список выражений, EStructLit использует RustPath и отображение полей в Expr. Следовательно, выражения образуют уже не просто дерево, а узел взаимной рекурсии между типами, путями и телами функций.

С выражениями тесно связан тип Pattern. Он также рекурсивен: паттерн может быть переменной, литералом, кортежем, структурным паттерном, вариантом перечисления или модифицированным паттерном mut. Внутри Pattern снова появляются Literal и RustPath. Это особенно важно для конструкции match, где в одной точке модели сходятся выражения, паттерны и блоки. Соответствующий тип MatchArm связывает паттерн, опциональное guard-выражение и результирующее выражение, то есть фиксирует один из типичных узлов синтаксиса Rust как отношение между несколькими рекурсивными сущностями модели.

Для описания тел функций используются Statement и Block. Здесь модель снова остаётся рекурсивной: оператор SExpr содержит выражение, SLet содержит паттерн, опциональный RustType и опциональное выражение, а Block есть список операторов. Поскольку выражение может содержать блок (EBlock), а блок состоит из операторов, включающих выражения, возникает естественная взаимная рекурсия между уровнями *expression*

и *statement*. Без такой структуры невозможно корректно выразить идиоматический синтаксис Rust, где блоки сами являются выражениями.

На более служебном уровне модель также фиксирует литералы, времена жизни, модификаторы видимости, операторы, self-аргументы и импорты. Эти типы менее сложны, однако они выполняют важную нормализующую функцию: позволяют представлять даже небольшие синтаксические различия не строковыми соглашениями, а конечными множествами вариантов. Так, тип *UseTree* выражает дерево импорта, *Visibility* отделяет различные уровни видимости, а *Lifetime* делает времена жизни частью общей модели типа, а не внешней текстовой аннотацией.

Следовательно, модель языка Rust в проекте представляет собой не набор вспомогательных структур, а формальную внутреннюю грамматику подмножества Rust, достаточного для генерации модулей словаря объектов. Её рекурсивность отражает рекурсивность самого языка, а взаимная рекурсия между типами, путями, выражениями, паттернами и блоками позволяет выражать синтаксис не постфактум через печать строк, а конструктивно, на уровне типов Haskell.

6.4 Двухуровневый переводчик

6.4.1 Назначение

Задача двухуровневого переводчика состоит в отображении одного объекта *CompleteObject* во внутреннее Rust-представление. Концептуально на обоих уровнях перевода выполняется одна и та же последовательность действий:

1. строится представление Rust-типа;
2. к нему добавляются необходимые *trait*-реализации;
3. к нему добавляются требуемые атрибуты.

Однако тождество общей схемы не означает тождества механики. На верхнем уровне переводчик работает с объектом как с целостной единицей: различает *VAR*, *ARRAY* и *RECORD*, определяет его главную форму в языке Rust и вычисляет зависимости. На нижнем уровне переводчик работает уже с более

локальными компонентами: отдельной записью `UnifiedEntry`, типом элемента массива, полем структуры или модифицированной через REG записью. Следовательно, действия на обоих уровнях изоморфны по схеме, но различны по предмету обработки и набору условий.

Именно эта асимметрия и требует двухуровневой архитектуры. Если попытаться описать весь перевод в одном слое, то логика верхнеуровневой формы объекта, обработка зависимостей, учёт модификаций REG, генерация вложенных типов и добавление trait-ов смешаются в одном наборе функций. Разделение на два уровня позволяет локализовать разные классы решений: верхний уровень отвечает за композицию законченного объекта, нижний — за перевод его элементарных составляющих.

6.4.2 Архитектура уровней

Верхний уровень переводчика реализован в модулях семейства `Rust.Translator.First.*`. Его входом служит `CompleteObject`, то есть уже свёрнутый объект прикладной модели. На этом уровне определяется, какую верхнюю форму получит объект в Rust. Если обрабатывается `CompVar`, перевод делегируется нижнему уровню почти без дополнительной композиции. Если обрабатывается `CompArray`, верхний уровень строит массив фиксированной длины и при необходимости добавляет зависимость для типа элемента. Если обрабатывается `CompStruct`, верхний уровень формирует структуру и организует перевод её полей как зависимостей.

Нижний уровень переводчика реализован в модулях `Rust.Translator.Second.*`. Его входом служит `UnifiedEntry`, то есть либо чистая запись словаря объектов, либо запись, дополненная интерпретацией REG. На этом уровне уже не решается вопрос о том, является ли объект массивом или структурой. Здесь решается другой вопрос: как именно должен быть выражен конкретный элемент в терминах языка Rust. Внутри нижнего уровня различаются два базовых случая:

- объект не модифицирован через REG и представляется как обычный тип, соответствующий типу данных `CANopen`;
- объект модифицирован через REG и должен быть развёрнут в более

выразительную форму представления.

На практике это означает, что запись словаря, имеющая в EDS тип `UNSIGNED16`, может превратиться либо в простой обёрточный тип над `u16`, либо в структуру флагов, либо в перечисление. Такой переход особенно важен для объектов управления и диагностики, где числовой код сам по себе мало что говорит прикладному программисту.

Тем самым различие между уровнями носит не количественный, а качественный характер. Верхний уровень отвечает за композицию целого объекта и организацию зависимостей. Нижний уровень отвечает за выбор конкретной языковой формы для элементарной записи. Иначе говоря, верхний уровень оперирует вариантами `CompVar`, `CompArray` и `CompStruct`, определяя правила перевода для законченного объекта, тогда как нижний уровень оперирует вариантами `Native` и `Modified`, а также базовыми типами, структурами полей и перечислениями. Благодаря этому каждый уровень остаётся относительно локальным по своей ответственности.

6.4.3 Почему существует `RustDesc`

С архитектурной точки зрения одним из ключевых решений данного этапа является введение типа `RustDesc`. Его определение имеет вид:

Листинг 18 – Тип `RustDesc`

```
1 data RustDesc = RustDesc RustEntry [RustDesc]
```

Необходимость этого типа определяется тем, что результат перевода одного объекта почти никогда не исчерпывается одной верхнеуровневой записью. Объект может порождать дополнительные типы-зависимости: например, поле структуры или модифицированный элемент массива может требовать собственного определения. Если бы переводчик возвращал только `RustEntry`, то зависимости пришлось бы передавать и хранить отдельно, а согласованность между основным элементом и порождёнными им типами обеспечивалась бы внешними соглашениями.

`RustDesc` устраняет эту проблему, удерживая в одном значении и сам

элемент, и все его зависимости. Это делает результат перевода локально замкнутым: всё, что необходимо для вывода данного элемента, уже находится рядом с ним. Благодаря такому решению этап печати может обрабатывать не разрозненные записи, а законченные фрагменты внутреннего синтаксического дерева, в которых верхний элемент и его зависимые элементы образуют единое целое.

6.4.4 Интерпретация Rust-типа

Построение верхнеуровневой формы объекта выполняет функция `buildC0From` из модуля `Rust.Translator.First.Base`. Её задача состоит в том, чтобы по значению `CompleteObject` определить, какой именно `RustEntry` должен стать главным элементом результата.

Листинг 19 – Тип `RustEntry`

```
1 data RustEntry = RustEntry
2   { _rustVis    :: Visibility
3     , _rustName  :: String
4     , _rustAttrs :: Attributes
5     , _rustRep   :: RustRep
6     , _rustImpls :: [ImplBlock]
7   }
```

Смысл функции `buildC0From` состоит в выборе значения поля `rustRep` и, тем самым, в интерпретации объекта как одной из допустимых форм Rust-представления. Для `CompVar` верхний уровень делегирует построение формы нижнему уровню. Для `CompArray` он строит массив заданной длины и, если элемент массива модифицирован, порождает зависимый тип элемента. Для `CompStruct` он формирует структуру и добавляет зависимости для тех полей, которые требуют самостоятельных определений. Следовательно, `buildC0From` является той точкой, где объект прикладной модели впервые получает свою верхнюю Rust-форму.

6.4.5 Задание черт

После того как форма объекта определена, верхний уровень переводчика добавляет к ней необходимые trait-реализации. За это отвечает функция `addTraits` из модуля `Rust.Translator.First.Traits`. Она анализирует уже построенное представление `RustRep` и в зависимости от него дополняет запись соответствующими `impl`-блоками.

Листинг 20 – Тип `ImplBlock`

```
1 data ImplBlock = ImplBlock
2   { implAttrs    :: Attributes
3     , implGenerics :: Generics
4     , implTrait   :: TraitRef
5     , implFor     :: RustType
6     , implItems   :: ImplItems
7   }
```

На верхнем уровне функция `addTraits` работает прежде всего со структурными формами объекта. Для массива она добавляет реализации, связанные с индексацией. Для структуры — реализации, связанные с поведением типа как составного значения. Для базовых и перечислимых форм значительная часть trait-логики уже делегирована нижнему уровню. Следовательно, `addTraits` не просто механически добавляет trait-ы, а распределяет ответственность между уровнями переводчика в зависимости от того, где именно возникает соответствующая семантика.

6.4.6 Задание атрибутов

Третьим шагом верхнего уровня является задание атрибутов, выполняемое функцией `addDerives` из модуля `Rust.Translator.First.Attrs`. Эта функция также ориентируется на уже выбранную форму `RustRep`, но, в отличие от `addTraits`, работает не с поведением типа, а с его метаописанием.

Листинг 21 – Тип `Attributes`

```
1 newtype Attributes = Attributes [Attribute]
2 data Attribute
3   = AttrDerive [RustPath]
```

На верхнем уровне `addDerives` прежде всего добавляет `derive`-атрибуты для структурных форм. Это означает, что атрибуты рассматриваются как отдельный слой метаинформации, который не следует смешивать ни с интерпретацией типа, ни с набором `trait`-реализаций. Такое разбиение делает архитектуру переводчика более прозрачной: форма типа задаётся через `buildC0From`, поведение — через `addTraits`, а метаописание — через `addDerives`.

6.5 Аннотирующие типы

6.5.1 Назначение

Особенностью проекта является генерация аннотирующих типов `Index`, `Value`, `Dict`, выполняемая модулем `Rust.Annotator`. Эти типы строятся только для модулей `Dictionary`. С прикладной точки зрения они образуют единый интерфейс доступа ко всему словарю: перечисление индексов, перечисление допустимых значений и структурное представление словаря как целостного объекта.

6.5.2 Архитектура генерации аннотирующих типов

С архитектурной точки зрения генерация аннотирующих типов представляет собой отдельную фазу поверх уже выполненного перевода объектов. Её входом служит не исходная карта `Compiled`, а согласованный контекст, в котором каждому индексу сопоставлены две взаимно согласованные представления: исходный объект `CompleteObject` и уже построенный результат `Rust`-перевода. Именно по этой причине в модуле используется тип `AnnCtx`, а функция `zipAnnCtx` сначала проверяет совпадение множеств индексов и только затем объединяет обе карты в единый контекст.

Результат этой фазы задаётся типом `AnnResult`:

```
1 data AnnResult = AnnResult
2   { _annotation :: Annotation
3     , _extraImpls :: Map Ind [ImplBlock]
4   }
```

Генерация организована как последовательность трёх операций над структурой `Annotation`. Сначала функция `initAnn` создаёт каркас трёх верхнеуровневых записей: `Index`, `Value` и `Dict`. Затем функция `fillAnn` заполняет их конкретным содержанием по карте уже переведённых объектов: `Index` получает перечисление имён объектов, `Value` — перечисление вариантов со связанными типами, `Dict` — структуру с полями словаря. После этого `traitAnn` добавляет необходимый набор `trait`-реализаций, а `deriveAnn` добавляет `derive`-атрибуты.

Для вычисления `trait`-ов используется вспомогательный тип `EntryInfo`. Он сводит разнородные сведения об объекте — имя, вариант в перечислении `Value`, имя поля в `Dict`, значение по умолчанию, тип данных и вид объекта — к единой форме, пригодной для шаблонной генерации `trait`-реализаций. Благодаря этому логика построения аннотирующих типов отделяется от конкретного способа хранения информации в `CompleteObject` и `RustDesc`.

6.5.3 Секции и словари

Принципиально важно, что аннотирующие типы строятся только для модулей типа `Dictionary`. Для `Section` такая генерация не выполняется. Причина состоит в том, что секция не является самостоятельным словарём и не должна порождать собственный унифицированный интерфейс доступа. Её задача — поставлять типы и переведённые объекты, которые затем могут быть включены в контекст итогового словаря.

Именно здесь раскрывается смысл двух полей `AnnResult`. Поле `annotation` содержит собственно три аннотирующих типа текущего словаря: `Index`, `Value` и `Dict`. Эти типы затем включаются в печатаемый модуль как обычные `RustEntry`. Поле `extraImpls`, напротив, содержит дополни-

тельные `impl`-блоки для уже существующих объектов словаря. Эти блоки необходимы для того, чтобы каждый конкретный тип объекта мог участвовать в общем интерфейсе словаря, например преобразовываться в `Value`, извлекаться из него и соотноситься с соответствующим индексом.

Для словаря генерация строится по расширенному контексту объектов. Текущий модуль предоставляет собственную карту переведённых описаний, а уже обработанные секции передаются в виде списка `Artifact`. Функция `zipArtifact` извлекает из каждого артефакта его переведённые описания и исходные объекты, после чего все такие контексты объединяются с контекстом текущего словаря. Благодаря этому аннотирующие типы строятся не по локальному фрагменту, а по полной совокупности объектов, доступных в `Dictionary`.

Такое решение согласуется и с механизмом импортов секций. Модуль словаря получает импорты вида `use crate::section::*`;, а на уровне аннотирующих типов использует не только имена этих секций, но и их уже построенные Rust-представления. Следовательно, секции участвуют в генерации словаря сразу на двух уровнях: как импортируемые модули и как источники объектов для построения единого интерфейса `Index – Value – Dict`.

6.6 Генерация импортов

После того как объекты переведены, аннотирующие типы построены, а дополнительные `impl`-блоки добавлены, этап Rust должен вычислить набор импортов, необходимых для корректной компиляции результирующего модуля. Эта задача не сводится к механическому добавлению нескольких строк `use`. Напротив, проект рассматривает импорт как часть внутренней модели модуля и потому вычисляет его систематически, используя тип `UseImport` и множества импортов.

Архитектурно генерация импортов устроена как объединение трёх различных источников:

1. обязательных импортов, жёстко заданных в программе;
2. условных импортов, вычисляемых по уже построенным объектам;
3. импортов секций, зависящих от структуры связей между модулями.

Обязательные импорты определены в модуле `Rust.Imports.Mandatory`. Они представляют собой фиксированное множество зависимостей, необходимых практически для любого генерируемого модуля. К ним относятся, например, импорт модуля `crate::error::*`, стандартные элементы `std::default::*` и `std::fmt::Debug`, `Formatter`, а также импорт интерфейсов библиотеки `funcan_rs`. Эти импорты не выводятся из конкретных объектов словаря, а встроены в архитектуру генератора как часть базовой инфраструктуры выходного кода.

Условные импорты вычисляются модулем `Rust.Imports.Conditional`. Их источник — уже построенное внутреннее описание Rust-объектов, то есть значения `RustDesc`. Функция `importsDesc` рекурсивно проходит по описанию объекта и его зависимостям, а затем, анализируя `RustEntry`, `RustRep` и вложенные `impl`-блоки, строит множество дополнительных импортов. В текущей реализации наиболее показателен случай числовых типов: если в представлении встречается арифметический `DataType`, генератор добавляет импорт `std::ops::*`. Для массивов отдельно учитывается необходимость импорта `std::ops::Index`. Тем самым условные импорты выражают зависимость между синтаксической формой сгенерированного объекта и теми внешними сущностями Rust, которые требуются для его корректной компиляции.

Третий класс образуют импорты секций, задаваемые модулем `Rust.Imports.Sections`. Их роль отличается от двух предыдущих случаев. Если обязательные импорты встроены в программу, а условные выводятся из формы объектов, то импорты секций зависят от модульной структуры словаря. Для каждого подключённого `Section` формируется импорт вида `use crate::section::*`; Таким образом, словарь получает доступ к типам и определениям, уже вынесенным в общие модули.

Итоговый набор импортов формируется в функции `manageImports` как объединение этих трёх множеств. Сначала из состояния извлекаются уже построенные Rust-описания объектов, затем из окружения берутся имена секций, после чего вычисляются: `cond` — условные импорты, `secs` — импорты секций и `defaultImports` — обязательное множество. Их объединение за-

писывается в поле `imports` состояния `RustState`.

Такое решение важно по двум причинам. Во-первых, оно устраняет дублирование и фрагментарность: импорты не распределены по коду печати и не накапливаются случайным образом, а вычисляются в одной локализованной фазе. Во-вторых, оно делает систему расширяемой. При появлении новых конструкций Rust или новых источников зависимостей достаточно расширить соответствующий класс импортов, не меняя общую архитектуру этапа.

6.7 Вывод модуля

Фаза вывода модуля завершает весь этап Rust. Если предыдущие шаги строили внутреннюю модель результата, то здесь эта модель материализуется в текст исходного кода и записывается в файл. Соответствующая логика сосредоточена в функции `generate`, а за собственно печать отвечают модули семейства `Rust.Printer.*`.

Прежде всего функция `generate` извлекает из окружения и состояния все компоненты, необходимые для финальной сборки модуля: каталог вывода и имя текущего модуля из `Desc`, множество импортов, карту переведённых описаний `descs`, карту дополнительных `impl`-блоков `exImpls` и список аннотирующих типов, извлекаемый из `Annotation`. Тем самым на этом шаге в одной точке собираются все результаты предыдущих фаз.

Далее выполняется важная операция `mergeImpls`. Её задача состоит в том, чтобы объединить дополнительные `impl`-блоки, сгенерированные, например, механизмом аннотирующих типов, с теми объектами, к которым они относятся. Если соответствующий `impl`-блок относится к уже существующему объекту модуля, он встраивается в его `RustDesc`. Если же он не принадлежит ни одному локальному объекту, то остаётся в отдельном наборе и печатается самостоятельным кластером. Благодаря этому итоговый вывод не теряет ни одной реализации `trait`-а и при этом сохраняет структурную связь между объектом и его поведением там, где такая связь существует.

Собственно генерация текста выполняется функцией `renderModule`. Она получает четыре группы сущностей:

- список импортов;

- список основных описаний `RustDesc`;
- список внешних кластеров `impl`-блоков;
- список аннотирующих `RustEntry`.

Модуль `Rust.Printer` не формирует текст напрямую конкатенацией строк, а использует библиотеку `Prettyprinter`. Это позволяет сначала представить модуль как композицию документов `Doc`, а уже затем отрендерить его в `Text`. Такой подход согласуется с общей архитектурой проекта: и на уровне синтаксической модели `Rust`, и на уровне печати используется структурное, а не строковое представление.

Печать организована поблочно. Функция `importsPP` формирует блок импортов. Функция `rustPP` печатает основные описания объектов, причём каждое значение `RustDesc` выводится рекурсивно вместе со своими зависимостями. Функция `implsPP` печатает внешние группы `impl`-блоков, а `annPP` печатает аннотирующие типы отдельным завершающим блоком. Таким образом, структура текста модуля непосредственно отражает структуру внутренней модели, а не возникает как побочный эффект разрознённых процедур печати.

После формирования текста остаётся выполнить материализацию в файловой системе. Функция `generate` проверяет существование каталога вывода и при необходимости создаёт его, затем вычисляет путь к выходному файлу по имени модуля и записывает туда сформированный `Text`. Тем самым вывод модуля завершает не только логический, но и физический цикл генерации: внутреннее представление превращается в файл `.rs`, который может быть непосредственно использован компилятором `Rust`.

С архитектурной точки зрения важно, что этап вывода не принимает собственных решений о семантике объектов. Все такие решения были приняты ранее. Фаза `generate` выполняет иную функцию: она сохраняет структурную согласованность уже построенной модели при переходе к внешнему текстовому представлению. Поэтому её следует рассматривать не как тривиальный финальный шаг, а как отдельную фазу конвейера, отвечающую за точную материализацию результата.

7 Пример работы генератора

Для демонстрации работы генератора рассмотрим пример, состоящий из одной переиспользуемой секции `Info` и одного словаря `Maxon`. Дескриптор секции приведён в листинге A.1, а дескриптор словаря — в листинге A.2. Из этих листингов видно, что секция `Info` строится на основе того же файла `EDS`, что и словарь, но выделяет лишь общую часть описания, тогда как словарь `Maxon` дополнительно подключает секцию через `Uses` и использует файл регистровых интерпретаций.

Содержательно секция `Info` представляет собой фрагмент словаря, содержащий общую информацию об устройстве `CANopen`. В рассматриваемом примере она включает объект `0x1000 Device type`, который относится к стандартным коммуникационным параметрам и должен присутствовать в словаре любого совместимого устройства [1]. Словарь `Maxon`, напротив, относится уже не к абстрактному устройству `CANopen`, а к конкретному семейству контроллеров `maxon EPOS4` [`maxon_epos4_control_2024`; 5].

Вспомогательные входные данные для этого примера приведены в приложении: фильтр секции показан в листинге A.3, фильтр словаря — в листинге A.4, файл регистровых интерпретаций — в листинге A.5, а фрагмент исходного словаря объектов — в листинге A.6. В рассматриваемом сценарии секция `Info` выделяет объект `0x1000`, а словарь `Maxon` сосредоточен на объекте `0x3000`. При этом файл `REG` задаёт дополнительную прикладную интерпретацию для записи `3000sub1`, позволяя представить её не как сырое числовое значение, а как систему перечислений и структурных полей.

Результат генерации подтверждает корректность такой схемы. Сгенерированный модуль секции `Info` приведён в листинге A.7: он содержит тип `DeviceType`, соответствующий объекту `0x1000`. Основной результат показан в листинге A.8, где модуль словаря `Maxon` уже импортирует секцию `Info`, содержит типы `SensorType1`, `SensorType2`, `SensorType3`, `SensorsConfiguration`, `ControlStructure`, `AxisConfiguration`, а также аннотирующие типы `MaxonIndex`, `MaxonValue` и `MaxonDict`. Тем самым на одном примере одновременно наблюдаются все ключевые свойства системы: выделение общей секции, фильтрация области генерации, приклад-

ная интерпретация словарных значений и построение итогового типизированного интерфейса на языке Rust.

ЗАКЛЮЧЕНИЕ

В работе был рассмотрен и реализован кодогенератор для построения типизированного Rust-интерфейса к словарю объектов CANopen. Исходная постановка задачи исходила из того, что одного только формального описания словаря в EDS недостаточно для удобной прикладной разработки: помимо адресации объектов необходимо учитывать область генерации, прикладную интерпретацию значений и повторное использование общих частей словаря. Именно эта совокупность требований определила архитектуру всей системы.

В ходе работы была проанализирована предметная область CANopen и показано, что данные словаря объектов естественно существуют сразу на нескольких уровнях интерпретации: протокольном, прикладном и уровне программного интерфейса. На этой основе были сформулированы требования к генератору, включающие поддержку нескольких входных форматов, декомпозицию на независимые этапы, возможность секционирования словаря, прикладную интерпретацию регистровых значений и генерацию целостного API целевого языка.

Ключевым результатом работы стала разработка поэтапной архитектуры, включающей *Init*, *Raw*, *Compile* и *Rust*. Для каждого из этих этапов были определены границы ответственности, характер входных и выходных данных, локализуемые классы ошибок и инварианты результата. Показано, что такая декомпозиция позволяет разделить конфигурационный анализ, синтаксический разбор, семантическую сборку промежуточной модели и генерацию выходного кода, не смешивая задачи разных уровней в одном наборе функций.

С архитектурной точки зрения существенным итогом является явное разделение входных контекстов этапов и их рабочих состояний. Такое решение позволило выразить границу между данными, уже подготовленными предыдущими вычислениями, и данными, которые формируются внутри текущего этапа. Для этапов, где требуется диагностика и работа с файловой системой, отдельно показана необходимость сохранения IO как части общей вычислительной среды.

На содержательном уровне центральную роль играет этап `Compile`, где сырые результаты разбора превращаются в прикладные объекты, пригодные для дальнейшего отображения в язык программирования. Следующим существенным результатом является разработка этапа `Rust`, включающего собственную внутреннюю модель языка, двухуровневый переводчик, механизм генерации аннотирующих типов и фазу систематического вычисления импортов. Это позволило перейти от набора разрозненных словарных записей к структурированному модулю, содержащему не только пользовательские типы, но и унифицированный интерфейс доступа к словарю через `Index`, `Value` и `Dict`.

Практический результат работы подтверждён на примере генерации модулей для секции `Info` и словаря `Махон`. Рассмотренный пример показывает, что система поддерживает повторное использование секций, корректно учитывает фильтрацию, использует файл `REG` для прикладной интерпретации словарных значений и формирует итоговый `Rust`-код, пригодный для дальнейшего применения в прикладной разработке.

Таким образом, поставленная цель разработки кодогенератора для работы с `CANopen` достигнута. Построенная система демонстрирует, что сочетание функциональной архитектуры на `Haskell` с формальной промежуточной моделью и структурной генерацией `Rust`-кода позволяет получать содержательный, расширяемый и типобезопасный результат. Дальнейшее развитие работы может быть связано с расширением поддерживаемых описаний объектов, углублением генерации для более сложных словарных структур и переносом построенного конвейера на другие целевые языки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. CiA 301 version 4.2.0: CANopen application layer and communication profile / CAN in Automation (CiA). — 02/21/2011. — URL: <https://www.can-cia.org/cia-groups/technical-documents> (visited on 04/22/2026).
2. *Falch M.* CANopen Explained - A Simple Intro / CSS Electronics. — 02/2025. — URL: <https://www.csselectronics.com/pages/canopen-tutorial-simple-intro> (visited on 12/24/2025).
3. CANopenNode: Object Dictionary / CANopenNode. — URL: https://canopennode.github.io/CANopenSocket/md_doc_objectDictionary.html (visited on 04/12/2026).
4. *Gedenk T.* CANopen Object Code / Embedded Communication Experts GmbH. — URL: <https://www.embedded-communication.com/en/canopen/canopen-object-code/> (visited on 04/12/2026).
5. EPOS4 Firmware Specification / maxon. — 2022. — URL: https://www.maxongroup.us/medias/sys_master/root/9444047912990/EP0S4-Firmware-Specification-En.pdf.
6. *Falch M.* CAN Bus Explained - A Simple Intro / CSS Electronics. — 01/2025. — URL: <https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial> (visited on 12/24/2025).
7. *O'Sullivan B., Stewart D., Goerzen J.* Real World Haskell. — O'Reilly Media, 2008. — ISBN 978-0-596-51498-3.
8. *Granin A.* Functional Design and Architecture. — Manning Publications, 2024. — ISBN 978-1-617-29961-2.
9. IO: The standard IO API / Hackage. — URL: <https://hackage-content.haskell.org/package/base/docs/System-IO.html> (visited on 04/12/2026).
10. except: Monad transformer to extend a monad with the ability to throw and catch exceptions / Hackage. — URL: <https://hackage-content.haskell.org/package/transformers-0.6.3.0/docs/Control-Monad-Trans-Except.html> (visited on 04/12/2026).
11. mtl: Monad classes, using functional dependencies / Hackage. — URL: <https://hackage.haskell.org/package/mtl> (visited on 04/12/2026).
12. *Hutton G., Meijer E.* Monadic Parsing in Haskell // Journal of Functional Programming. — 1998. — Vol. 8, no. 4. — P. 437–444. — DOI: 10.1017/S0956796898003050.

13. *Leijen D., Meijer E.* Parsec: Direct Style Monadic Parser Combinators For The Real World : tech. rep. / Universiteit Utrecht. — 2001. — UU-CS-2001-35. — URL: <https://www.microsoft.com/en-us/research/publication/parsec-direct-style-monadic-parser-combinators-for-the-real-world/> (visited on 04/12/2026).
14. parsec: Monadic parser combinators / Hackage. — URL: <https://hackage.haskell.org/package/parsec> (visited on 04/12/2026).
15. *McBride C., Paterson R.* Applicative Programming with Effects // Journal of Functional Programming. — 2008. — Vol. 18, no. 1. — P. 1–13. — DOI: 10.1017/S0956796807006326.
16. The Rust Reference: Grammar Summary / Rust Project Developers. — URL: <https://doc.rust-lang.org/reference/grammar.html> (visited on 04/22/2026).

Приложение А

Листинг А.1 – Дескриптор секции Info (info.desc)

```
1 Name = Info
2 Type = Section
3 EDS = maxon.eds
4 Filter = info.flt
```

Листинг А.2 – Дескриптор словаря Maxon (maxon.desc)

```
1 Name = Maxon
2 Type = dictionary
3 Uses = Info
4 EDS = maxon.eds
5 Filter = maxon.flt
6 Register = maxon.reg
```

Листинг А.3 – Фильтр секции Info (info.flt)

```
1 [1000]
```

Листинг А.4 – Фильтр словаря Maxon (maxon.flt)

```
1 [3000]
```

Листинг А.5 – Файл регистровых интерпретаций Maxon (maxon.reg)

```
1 [3000sub1]: {
2     0-7: Sensor type 1 {
3         Done = 0,
4         Digital incremental encoder 1 = 1
5     },
```

```

6      8-15: Sensor type 2 {
7          None = 0,
8          Digital incremental encoder 2 = 1,
9          Analog incremental encoder SinCos = 2,
10         SSI absolute encoder = 3
11     },
12     16-23: Sensor type 3 {
13         None = 0,
14         Digital Hall Sensor = 1
15     }
16 }

```

Листинг А.6 – Фрагмент словаря объектов Maxon (maxon.eds)

```

1  [1000]
2  ParameterName=Device type
3  ObjectType=0x7
4  DataType=0x7
5  AccessType=ro
6  DefaultValue=0x00020192
7  PDOMapping=0
8  ObjFlags=1

9  [3000]
10 SubNumber=7
11 ParameterName=Axis configuration
12 ObjectType=0x9

13 [3000sub0]
14 ParameterName=Highest sub-index supported
15 ObjectType=0x7
16 DataType=0x5
17 AccessType=ro
18 DefaultValue=6
19 PDOMapping=0
20 ObjFlags=1

21 [3000sub1]
22 ParameterName=Sensors configuration
23 ObjectType=0x7
24 DataType=0x7
25 AccessType=rw

```

```

26 DefaultValue=0x00100001
27 PDOMapping=0
28 ObjFlags=0

29 [3000sub2]
30 ParameterName=Control structure
31 ObjectType=0x7
32 DataType=0x7
33 AccessType=rw
34 DefaultValue=0x00010111
35 PDOMapping=0
36 ObjFlags=0

```

Листинг A.7 – Сгенерированный модуль Info (info.rs)

```

1 use crate::error::*;
2 use funcan_rs::dictionary::*;
3 use funcan_rs::interfaces::*;
4 use std::default::*;
5 use std::fmt::{Debug, Formatter};
6 use std::ops::*;

7 //-----Device type-----

8 #[derive(Clone, Copy)]
9 pub struct DeviceType(pub u32);

10 impl Debug for DeviceType {
11     fn fmt(&self, f: &mut Formatter<'_>) -> std::fmt::Result {
12         write!(f, "{}", self.0)
13     }
14 }

15 impl Default for DeviceType {
16     fn default() -> Self {
17         Self(0x00020192)
18     }
19 }

```

Листинг A.8 – Сгенерированный модуль Maxon (maxon.rs)

```
1 use crate::error::*;
2 use crate::info::*;
3 use funcan_rs::dictionary::*;
4 use funcan_rs::interfaces::*;
5 use std::default::*;
6 use std::fmt::{Debug, Formatter};

7 //-----Axis configuration-----

8 pub enum SensorType1 {
9     Done = 0,
10    DigitalIncrementalEncoder1 = 1
11 }

12 impl Default for SensorType1 {
13     fn default() -> Self {
14         Self::from(0)
15     }
16 }

17 impl Into<u32> for SensorType1 {
18     fn into(self) -> u32 {
19         self as u32
20     }
21 }

22 impl From<u32> for SensorType1 {
23     fn from(value: u32) -> Self {
24         match value {
25             0 => Self::Done,
26             1 => Self::DigitalIncrementalEncoder1,
27             _ => unreachable!('Невозможное значение')
28         }
29     }
30 }

31 pub enum SensorType2 {
32     None = 0,
33     DigitalIncrementalEncoder2 = 1,
34     AnalogIncrementalEncoderSincos = 2,
```

```

35     SsiAbsoluteEncoder = 3
36 }

37 impl Default for SensorType2 {
38     fn default() -> Self {
39         Self::from(0)
40     }
41 }

42 impl Into<u32> for SensorType2 {
43     fn into(self) -> u32 {
44         self as u32
45     }
46 }

47 impl From<u32> for SensorType2 {
48     fn from(value: u32) -> Self {
49         match value {
50             0 => Self::None,
51             1 => Self::DigitalIncrementalEncoder2,
52             2 => Self::AnalogIncrementalEncoderSincos,
53             3 => Self::SsiAbsoluteEncoder,
54             _ => unreachable!("Невозможное значение")
55         }
56     }
57 }

58 pub enum SensorType3 {
59     None = 0,
60     DigitalHallSensor = 1
61 }

62 impl Default for SensorType3 {
63     fn default() -> Self {
64         Self::from(0)
65     }
66 }

67 impl Into<u32> for SensorType3 {
68     fn into(self) -> u32 {
69         self as u32
70     }
71 }

```

```

72 impl From<u32> for SensorType3 {
73     fn from(value: u32) -> Self {
74         match value {
75             0 => Self::None,
76             1 => Self::DigitalHallSensor,
77             _ => unreachable!('Невозможное значение')
78         }
79     }
80 }

81 #[derive(Clone, Copy, Debug, Eq, PartialEq)]
82 pub struct SensorsConfiguration {
83     pub sensor_type_1: SensorType1,
84     pub sensor_type_2: SensorType2,
85     pub sensor_type_3: SensorType3
86 }

87 impl Default for SensorsConfiguration {
88     fn default() -> Self {
89         Self::from(0x00100001)
90     }
91 }

92 impl Into<u32> for SensorsConfiguration {
93     fn into(self) -> u32 {
94         let mut value = 0;
95         value |= (Into::<u32>::into(self.sensor_type_1) & 0xff) << 0;
96         value |= (Into::<u32>::into(self.sensor_type_2) & 0xff) << 8;
97         value |= (Into::<u32>::into(self.sensor_type_3) & 0xff) << 16;
98         value
99     }
100 }

101 impl From<u32> for SensorsConfiguration {
102     fn from(value: u32) -> Self {
103         Self {
104             sensor_type_1: SensorType1::from((value >> 0) & 0xff),
105             sensor_type_2: SensorType2::from((value >> 8) & 0xff),
106             sensor_type_3: SensorType3::from((value >> 16) & 0xff),
107         }
108     }
109 }

```

```

110 pub enum CurrentControlStructure {
111     PiCurrentController = 1
112 }

113 impl Default for CurrentControlStructure {
114     fn default() -> Self {
115         Self::from(0)
116     }
117 }

118 impl Into<u32> for CurrentControlStructure {
119     fn into(self) -> u32 {
120         self as u32
121     }
122 }

123 impl From<u32> for CurrentControlStructure {
124     fn from(value: u32) -> Self {
125         match value {
126             1 => Self::PiCurrentController,
127             _ => unreachable!('Невозможное значение')
128         }
129     }
130 }

131 pub enum VelocityControlStructure {
132     None = 0,
133     PiVelocityControllerLowPassFilter = 1,
134     PiVelocityControllerObserver = 2
135 }

136 impl Default for VelocityControlStructure {
137     fn default() -> Self {
138         Self::from(0)
139     }
140 }

141 impl Into<u32> for VelocityControlStructure {
142     fn into(self) -> u32 {
143         self as u32
144     }
145 }

```

```

146 impl From<u32> for VelocityControlStructure {
147     fn from(value: u32) -> Self {
148         match value {
149             0 => Self::None,
150             1 => Self::PiVelocityControllerLowPassFilter,
151             2 => Self::PiVelocityControllerObserver,
152             _ => unreachable!("Невозможное значение")
153         }
154     }
155 }

156 pub enum PositionControlStructure {
157     None = 0,
158     PidPositionController = 1,
159     DualLoopPositionController = 2
160 }

161 impl Default for PositionControlStructure {
162     fn default() -> Self {
163         Self::from(0)
164     }
165 }

166 impl Into<u32> for PositionControlStructure {
167     fn into(self) -> u32 {
168         self as u32
169     }
170 }

171 impl From<u32> for PositionControlStructure {
172     fn from(value: u32) -> Self {
173         match value {
174             0 => Self::None,
175             1 => Self::PidPositionController,
176             2 => Self::DualLoopPositionController,
177             _ => unreachable!("Невозможное значение")
178         }
179     }
180 }

181 pub enum Gear {
182     None = 0,

```



```

183     GearMounted = 1
184 }

185 impl Default for Gear {
186     fn default() -> Self {
187         Self::from(0)
188     }
189 }

190 impl Into<u32> for Gear {
191     fn into(self) -> u32 {
192         self as u32
193     }
194 }

195 impl From<u32> for Gear {
196     fn from(value: u32) -> Self {
197         match value {
198             0 => Self::None,
199             1 => Self::GearMounted,
200             _ => unreachable!('Невозможное значение')
201         }
202     }
203 }

204 pub enum ProcessValueReference {
205     OnMotor = 0,
206     OnGear = 1
207 }

208 impl Default for ProcessValueReference {
209     fn default() -> Self {
210         Self::from(0)
211     }
212 }

213 impl Into<u32> for ProcessValueReference {
214     fn into(self) -> u32 {
215         self as u32
216     }
217 }

218 impl From<u32> for ProcessValueReference {

```

```

219     fn from(value: u32) -> Self {
220         match value {
221             0 => Self::OnMotor,
222             1 => Self::OnGear,
223             _ => unreachable!('Невозможное значение')
224         }
225     }
226 }

227 pub enum MainSensor {
228     None = 0,
229     Sensor1 = 1,
230     Sensor2 = 2,
231     Sensor3 = 3
232 }

233 impl Default for MainSensor {
234     fn default() -> Self {
235         Self::from(0)
236     }
237 }

238 impl Into<u32> for MainSensor {
239     fn into(self) -> u32 {
240         self as u32
241     }
242 }

243 impl From<u32> for MainSensor {
244     fn from(value: u32) -> Self {
245         match value {
246             0 => Self::None,
247             1 => Self::Sensor1,
248             2 => Self::Sensor2,
249             3 => Self::Sensor3,
250             _ => unreachable!('Невозможное значение')
251         }
252     }
253 }

254 pub enum AuxiliarySensor {
255     None = 0,
256     Sensor1 = 1,

```

```

257     Sensor2 = 2,
258     Sensor3 = 3
259 }

260 impl Default for AuxiliarySensor {
261     fn default() -> Self {
262         Self::from(0)
263     }
264 }

265 impl Into<u32> for AuxiliarySensor {
266     fn into(self) -> u32 {
267         self as u32
268     }
269 }

270 impl From<u32> for AuxiliarySensor {
271     fn from(value: u32) -> Self {
272         match value {
273             0 => Self::None,
274             1 => Self::Sensor1,
275             2 => Self::Sensor2,
276             3 => Self::Sensor3,
277             _ => unreachable!('Невозможное значение')
278         }
279     }
280 }

281 pub enum MountingPositionSensor2 {
282     OnMotor = 0,
283     OnGear = 1
284 }

285 impl Default for MountingPositionSensor2 {
286     fn default() -> Self {
287         Self::from(0)
288     }
289 }

290 impl Into<u32> for MountingPositionSensor2 {
291     fn into(self) -> u32 {
292         self as u32
293     }

```

```

294 }

295 impl From<u32> for MountingPositionSensor2 {
296     fn from(value: u32) -> Self {
297         match value {
298             0 => Self::OnMotor,
299             1 => Self::OnGear,
300             _ => unreachable!("Невозможное значение")
301         }
302     }
303 }

304 #[derive(Clone, Copy, Debug, Eq, PartialEq)]
305 pub struct ControlStructure {
306     pub current_control_structure: CurrentControlStructure,
307     pub velocity_control_structure: VelocityControlStructure,
308     pub position_control_structure: PositionControlStructure,
309     pub gear: Gear,
310     pub process_value_reference: ProcessValueReference,
311     pub main_sensor: MainSensor,
312     pub auxiliary_sensor: AuxiliarySensor,
313     pub mounting_position_sensor_2: MountingPositionSensor2
314 }

315 impl Default for ControlStructure {
316     fn default() -> Self {
317         Self::from(0x00010111)
318     }
319 }

320 impl Into<u32> for ControlStructure {
321     fn into(self) -> u32 {
322         let mut value = 0;
323         value |= (Into::<u32>::into(self.current_control_structure) & 0xf) << 0;
324         value |= (Into::<u32>::into(self.velocity_control_structure) & 0xf) << 4;
325         value |= (Into::<u32>::into(self.position_control_structure) & 0xf) << 8;
326         value |= (Into::<u32>::into(self.gear) & 0x1) << 12;
327         value |= (Into::<u32>::into(self.process_value_reference) & 0x3) << 14;
328         value |= (Into::<u32>::into(self.main_sensor) & 0xf) << 16;
329         value |= (Into::<u32>::into(self.auxiliary_sensor) & 0xf) << 20;
330         value |= (Into::<u32>::into(self.mounting_position_sensor_2) & 0x3) << 26;
331         value
332     }

```

```

333 }

334 impl From<u32> for ControlStructure {
335     fn from(value: u32) -> Self {
336         Self {
337             current_control_structure: CurrentControlStructure::from((value >> 0) & 0),
338             velocity_control_structure: VelocityControlStructure::from((value >> 4) & 0),
339             position_control_structure: PositionControlStructure::from((value >> 8) & 0),
340             gear: Gear::from((value >> 12) & 0x1),
341             process_value_reference: ProcessValueReference::from((value >> 14) & 0x3),
342             main_sensor: MainSensor::from((value >> 16) & 0xf),
343             auxiliary_sensor: AuxiliarySensor::from((value >> 20) & 0xf),
344             mounting_position_sensor_2: MountingPositionSensor2::from((value >> 26) & 0xf),
345         }
346     }
347 }

348 #[derive(Clone, Copy, Debug, Eq, PartialEq)]
349 pub struct AxisConfiguration {
350     pub highest_sub_index_supported: u8,
351     pub sensors_configuration: SensorsConfiguration,
352     pub control_structure: ControlStructure
353 }

354 impl Default for AxisConfiguration {
355     fn default() -> Self {
356         Self {
357             highest_sub_index_supported: 0,
358             sensors_configuration: SensorsConfiguration::default(),
359             control_structure: ControlStructure::default(),
360         }
361     }
362 }

363 impl DictionaryValue<MaxonDict> for AxisConfiguration {
364     fn index() -> MaxonIndex {
365         MaxonIndex::AxisConfiguration
366     }
367 }

368 impl TryFrom<MaxonValue> for AxisConfiguration {
369     type Error = MaxonError;

```

```

370     fn try_from(v: MaxonValue) -> Result<Self, Self::Error> {
371         match v {
372             MaxonValue::AxisConfiguration(x) => Ok(x),
373             _ => Err(MaxonError::ValueMismatch)
374         }
375     }
376 }

377 impl Into<MaxonValue> for AxisConfiguration {
378     fn into(self: Self) -> MaxonValue {
379         MaxonValue::AxisConfiguration(self)
380     }
381 }

382 //-----DeviceType-----

383 impl DictionaryValue<MaxonDict> for DeviceType {
384     fn index() -> MaxonIndex {
385         MaxonIndex::DeviceType
386     }
387 }

388 impl TryFrom<MaxonValue> for DeviceType {
389     type Error = MaxonError;

390     fn try_from(v: MaxonValue) -> Result<Self, Self::Error> {
391         match v {
392             MaxonValue::DeviceType(x) => Ok(DeviceType(x)),
393             _ => Err(MaxonError::ValueMismatch)
394         }
395     }
396 }

397 impl Into<MaxonValue> for DeviceType {
398     fn into(self: Self) -> MaxonValue {
399         MaxonValue::DeviceType(self.0)
400     }
401 }

402 //-----Annotation-----

403 #[derive(Clone, Copy, Debug, Eq, Hash, PartialEq)]
404 pub enum MaxonIndex {
405     DeviceType,

```

```

406     AxisConfiguration
407 }

408 impl Into<CanIndex> for MaxonIndex {
409     fn into(self: Self) -> CanIndex {
410         match self {
411             MaxonIndex::DeviceType => CanIndex {base: 0x1000, sub: 0x0},
412             MaxonIndex::AxisConfiguration => CanIndex {base: 0x3000, sub: 0x0}
413         }
414     }
415 }

416 impl TryFrom<CanIndex> for MaxonIndex {
417     type Error = MaxonError;

418     fn try_from(ix: CanIndex) -> Result<Self, Self::Error> {
419         match (ix.base, ix.sub) {
420             (0x1000, 0x0) => Ok(MaxonIndex::DeviceType),
421             (0x3000, 0x0) => Ok(MaxonIndex::AxisConfiguration),
422             _ => Err(MaxonError::UnknownIndex(ix))
423         }
424     }
425 }

426 impl CanSize for MaxonIndex {
427     fn can_size(self: &Self) -> usize {
428         match self {
429             MaxonIndex::DeviceType => 4,
430             MaxonIndex::AxisConfiguration => todo!()
431         }
432     }
433 }

434 #[derive(Debug)]
435 pub enum MaxonValue {
436     DeviceType(u32),
437     AxisConfiguration(AxisConfiguration)
438 }

439 impl<'a> TryFrom<(MaxonIndex, &'a [u8])> for MaxonValue {
440     type Error = MaxonError;

441     fn try_from((ix, data): (MaxonIndex, &'a [u8])) -> Result<Self, Self::Error> {

```

```

442         match ix {
443             MaxonIndex::DeviceType => {
444                 let array: [u8; 4] = data.try_into()?;
445                 Ok(MaxonValue::DeviceType(u32::from_le_bytes(array)))
446             },
447             MaxonIndex::AxisConfiguration => todo!()
448         }
449     }
450 }

451 impl IntoBuf for MaxonValue {
452     fn into_buf<'a>(self:&'a Self, buf: &'a mut [u8]) -> usize {
453         let data: &[u8] = match self {
454             MaxonValue::DeviceType(v) => &v.to_le_bytes(),
455             MaxonValue::AxisConfiguration(v) => todo!()
456         };
457
458         let n = data.len();
459
460         assert!(buf.len() >= n);
461         buf[0..n].copy_from_slice(data);
462         n
463     }
464 }

465 pub struct MaxonDict {
466     pub device_type: u32,
467     pub axis_configuration: AxisConfiguration
468 }

469 impl Default for MaxonDict {
470     fn default() -> Self {
471         Self {device_type: 0x00020192, axis_configuration: AxisConfiguration::default}
472     }
473 }

474 impl Dictionary for MaxonDict {
475     type Index = MaxonIndex;
476
477     type Object = MaxonValue;
478
479     fn set(self: &mut Self, x: MaxonValue) {
480         match x {

```



```

477         MaxonValue::DeviceType(v) => {
478             self.device_type = v;
479         },
480         MaxonValue::AxisConfiguration(v) => {
481             self.axis_configuration = v.into();
482         }
483     }
484 }

485 fn get(self: &Self, x: &MaxonIndex) -> MaxonValue {
486     match x {
487         MaxonIndex::DeviceType => MaxonValue::DeviceType(self.device_type),
488         MaxonIndex::AxisConfiguration => MaxonValue::AxisConfiguration(self.axis_
489     }
490 }
491 }

```
